Resumen 1er Parcial ISW - 2026 2C

Temario

Temario....1
Unidad 1: Ingeniería de software en contexto....3
Introducción a la Ingeniería del Software. ¿Qué es? (PPT 1)....3
Estado Actual y Antecedentes. La Crisis del Software....4
Disciplinas que conforman la Ingeniería de Software....4
Ejemplos de grandes proyectos de software fallidos y exitosos....5
Ciclos de vida (Modelos de Proceso) y su influencia en la Administración de Proyectos de Software....5
Relación entre el ciclo de vida del proyecto y del producto....6
Clasificación de los ciclos de vida....6
Ventajas y desventajas de c/u de los ciclos de vida....8
Criterios para la selección de un modelo de proceso....8
Ejemplos de modelos de ciclo de vida (11)....8
Ciclos de vida (Modelos de Proceso) y Procesos de Desarrollo de Software....10
Procesos de Desarrollo Empíricos vs. Definidos....11
Componentes de un Proyecto de Sistemas de Información (PPT 6)....12
Proyecto....12
Administración de proyecto de software....13
Triple restricción....13
Líder de proyecto....14
Equipo de proyecto....15
Stakeholders....15
Plan de proyecto y sus 8 actividades....15
Factores para el éxito o fracaso....19
Vinculo proceso-proyecto-producto en la gestión de un proyecto de desarrollo de software....19
Unidad 1 Anexo: No Silver Bullet....21
Unidad 2: Gestión Lean-Agile de Productos de Software....24
Requerimientos Ágiles (PPT 3)....24
4 valores ágiles....24
12 principios del manifiesto ágil....26
Triángulo ágil y comparación de enfoques....28

Requerimientos ágiles....28
User Stories....32
Partes de una User Stories - Triple C:....32
Modelo INVEST....35
Diferentes Niveles de abstracción....35
Estimaciones Ágiles y de SW (PPT 4 - 7)....36
Técnicas de estimación....37
Problemas y errores en estimaciones....38
Estimación en ambientes ágiles....38
Planificación....40
Poker estimation....41
Diferencias con las estimaciones en ambientes tradicionales:....42
Gestión de Producto (PPT 5)....43
Evolución de los productos de software....44
Mínimo Producto Viable (MVP)....44
Design Thinking....45
Explicación MVP, MVF, MMF con el dinosaurio Agile....46
Comparación entre MVP, MMF, MVF, MRF y MLP....48
Framework Scrum (PPT 8)....49
Scrum Team....50
4 eventos de Scrum....51
3 artefactos de Scrum....52
Herramientas de scrum....54
Planificaciones en SCRUM....55
Unidad 3: Gestión del Software como producto....57
SCM (PPT 2)....57
Conceptos generales....58
Actividades fundamentales del SCM....61
Identificación de ítems de configuración....61
Control de cambios....61
Auditorías de configuración....62
Informe de estado....63
Evolución de la gestión de configuración de software....64
Gestión de configuración en ambientes ágiles....64

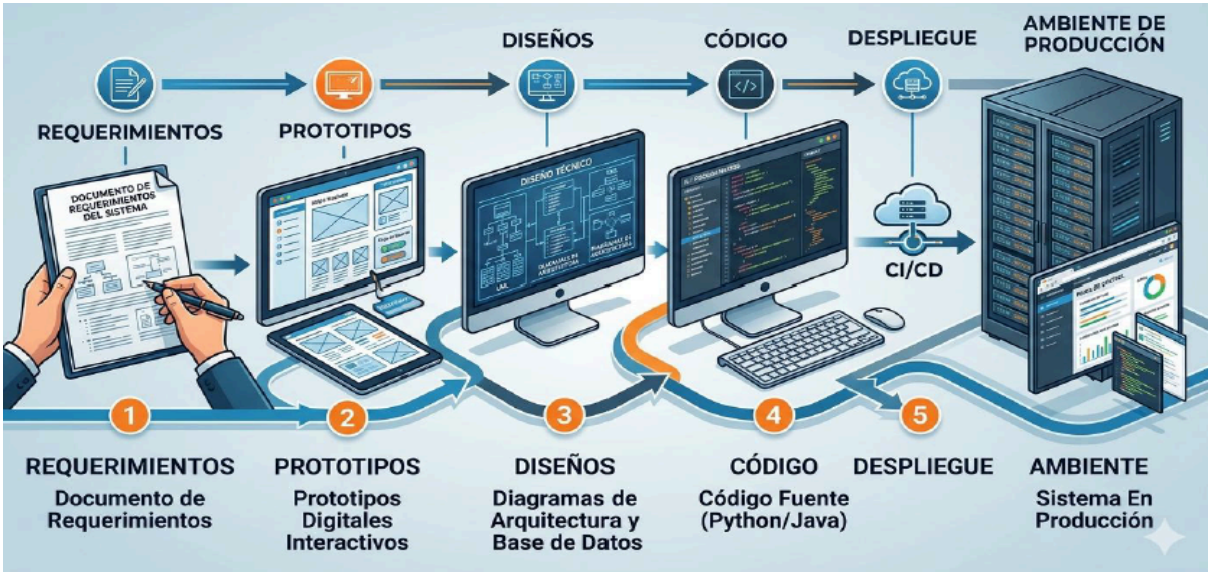
Unidad 1: Ingeniería de software en contexto

Introducción a la Ingeniería del Software. ¿Qué es? (PPT 1)

Software: Es un conjunto de programas en conjunto con toda la documentación requerida para definir, desarrollar y mantener los programas ejecutables que se entregan al cliente, como así también las herramientas utilizadas para la construcción de este.
Otra forma de definir el software es como, la información o conocimiento que se presenta en distintos niveles de abstracción, desde los requerimientos del sistema (abstracto), hasta el código que ejecuta el mismo (detallado).
Tipos de Software:

- **De Sistemas:** Se encarga de controlar y gestionar los recursos de hw de una computadora, así como brindar una interfaz para la interacción de los usuarios.
- **Utilitarios:** Referido al subconjunto de software del sistema que brinda utilidades/herramientas adicionales para mejorar la funcionalidad del SO y el rendimiento de la computadora (hw).
- **De aplicaciones:** Es utilizado para realizar tareas específicas en una computadora, por lo que está enfocado en satisfacer las necesidades de los usuarios → Enfoque de la materia.

Ingeniería de software: Es una disciplina que se encarga de todos los aspectos de la producción del software, como lo son:



Definición del IEEE: “La ingeniería de software es la aplicación de un enfoque sistemático, disciplinado y cuantificable al desarrollo, operación y mantenimiento del software; es decir, la aplicación de la ingeniería al software".
Sommerville: "La ingeniería de software es una disciplina de la ingeniería que se ocupa de todos los aspectos de la producción de software, desde la concepción inicial del sistema hasta su operación y mantenimiento".

Estado Actual y Antecedentes. La Crisis del Software.

La ingeniería de software nace tras las crisis del software, en la cual existía (y existe) un conjunto de dificultades o errores ocurridos en la planificación, estimación de los costos, productividad y calidad de un software, debido, principalmente, a la baja eficacia que presentan una gran cantidad de empresas al momento de desarrollarlo.
El término crisis del software hace referencia a un conjunto de hechos relativos al software, planteados en la conferencia de OTAN en 1968 por Friedrich Bauer, quien recalcó la dificultad para generar software libre de defectos, fácilmente comprensibles y que sean verificables. Sin embargo, este término fue utilizado anteriormente por Dijkstra en El Humilde Programador. Analizaremos las **causas de esta crisis**:

- La evolución de la tecnología del hardware mediante los circuitos integrados permitió la posibilidad del desarrollo de sistemas más grandes y complejos.
- En combinación con lo anterior, la demanda creciente de estos sistemas complejos, la subestimación de la complejidad que supone el desarrollo de software, los cambios a los que tienen que ser sometidos los productos para ser adaptados a las necesidades del cliente y la falta de una disciplina que intervenga en todos los aspectos de la producción del software fueron las causas de esta crisis.

En la actualidad, muchos de los sistemas desarrollados fracasan debido a fallas en el software, que principalmente son consecuencia de dos factores:

- **Demandas crecientes:** A medida que la tecnología y las técnicas de desarrollo evolucionan, la demanda de sistemas más grandes y complejos aumenta. Los usuarios necesitan que se construyan y distribuyan de manera rápida. La problemática aquí es que los métodos existentes (tradicionales) de ingeniería no permiten lograr esta “espontaneidad”, por lo que se deben desarrollar y poner a prueba nuevas técnicas y métodos de gestión y desarrollo.
- **Bajas expectativas:** Muchas de las compañías actuales de desarrollo de software no utilizan métodos de ingeniería de software en su trabajo diario. Por lo tanto, su software con frecuencia es más costoso y menos confiable de lo que debiera. La consecuencia, recae en la conformidad de las empresas y de los usuarios ante un software, es decir, las empresas se conforman con que el software solo “funcione”, sin tener en cuenta el valor agregado que el software le debe aportar al negocio del usuario. Es necesaria una mejor educación y capacitación en ingeniería de software para solucionar este problema.

Disciplinas que conforman la Ingeniería de Software.

Se divide en tres disciplinas:

- **Disciplinas técnicas:** Actividades que aportan al desarrollo de software como producto, como la toma y análisis de requerimientos, diseño, implementación, prueba, despliegue y documentación de software o capacitaciones a usuarios
- **Disciplinas de gestión:** Actividades de planificación, monitoreo y control del proyecto de desarrollo de software. Se utilizan, por ejemplo, metodologías LEAN Agile de gestión de proyectos.

- **Disciplinas de soporte:** Son transversales a los procesos de software, que permiten verificar la integridad y calidad de un producto de software. Entre ellas se incluyen la gestión de Configuración del Software (SCM), toma de métricas (ya sea para proceso, proyecto o producto) y aseguramiento de calidad (QA - Quality Assurance)

Ejemplos de grandes proyectos de software fallidos y exitosos.

Problemas comunes en el desarrollo:

- La versión final del producto no satisface las necesidades del cliente.
- No es fácil extenderlo y/o adaptarlo. Agregar más funcionalidad en otra versión es casi una misión imposible.
- Mala documentación y calidad.
- Más tiempos y costos que los presupuestados.

Factores que influyen en el fracaso de un producto de software:

1. Requerimientos incompletos 13.1 %
2. Falta de involucramiento del usuario 12.4 %
3. Falta de recursos 10.6 %
4. Expectativas poco realistas 9.3 %
5. Falta de apoyo de la Gerencia 8.7 %
6. Requerimientos cambiantes 8.1 %

Por estos factores, el 60% del software es desechado.

Factores que influyen en el éxito de un producto de software:

1. Involucramiento del usuario 15.9 %
2. Apoyo de la Gerencia 13.0 %
3. Enunciado claro de los requerimientos 9.6 %
4. Planeamiento adecuado 8.2 %
5. Expectativas realistas 7.7 %
6. Hitos intermedios 7.7 %
7. Personas involucradas competentes 7.2 %

Por eso es tan importante un ciclo iterativo para tener feedback temprano.

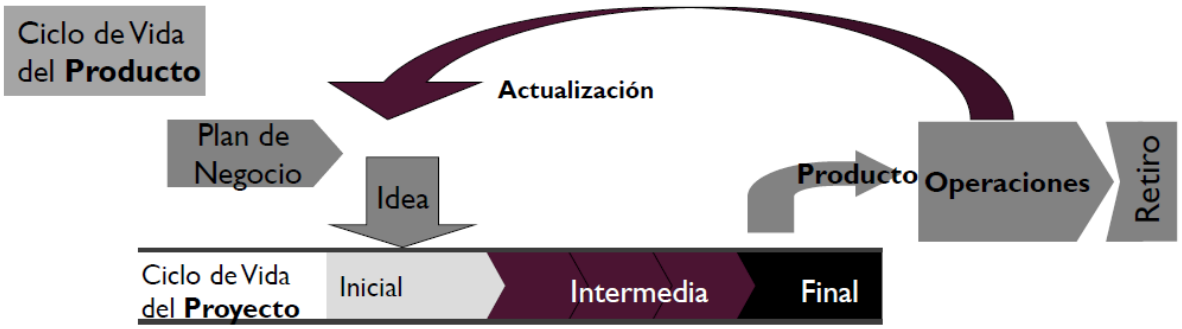
Ciclos de vida (Modelos de Proceso) y su influencia en la Administración de Proyectos de Software.

Se denomina ciclo de vida a una serie de pasos a través de los cuales un producto o un proyecto progresa en el tiempo. Es decir, que es una representación simplificada de un proceso, el cuál define elementos del proceso (actividades estructurales, productos del trabajo, tareas, etc.) y el flujo del proceso (o flujo de trabajo), que especifica la relación y el orden de dichos elementos. Es decir, son modelos genéricos de los procesos de software; y establecen los criterios que utiliza para determinar si se debe pasar de una tarea a la siguiente. Y son independientes de los procedimientos de cada actividad del ciclo de vida.

Los modelos de proceso pueden incluir las mismas actividades, pero cada uno, desde una perspectiva particular, pondrá más énfasis en algunas respecto a otras y definirán flujos de procesos diferentes. Además, establece los criterios que se utilizan para determinar cuándo se procede de una tarea a otra. Estos modelos especifican las fases y el orden para llevarlas a cabo.

Relación entre el ciclo de vida del proyecto y del producto

No existe un impacto entre los ciclos de vida de cada uno, sino que el ciclo de vida del producto siempre es mayor que el ciclo de vida del proyecto, ya que el ciclo de vida del proyecto dura lo que dura el desarrollo del software. En cambio, el ciclo de vida del producto dura hasta que el software se deje de utilizar, dependiendo esto de la madurez que tenga el producto en base a las necesidades del mercado.
Es posible que un producto tenga varios proyectos en su ciclo de vida, debido a constantes cambios y/o actualizaciones que se vayan realizando. Por lo que, dentro del ciclo de vida del producto, se pueden desarrollar varios ciclos de vida de proyectos.



¿Qué relación hay entre procesos de desarrollo y ciclos de vida?: Un proceso de desarrollo de software es el conjunto de actividades, tareas y prácticas que se realizan para desarrollar y mantener un producto de software. El ciclo de vida del software establece cómo se organizan y relacionan esas actividades a lo largo de la vida del producto, definiendo su secuencia, iteración y forma de ejecución. Por lo tanto, un modelo de ciclo de vida es una forma de organizar y estructurar un proceso de desarrollo.

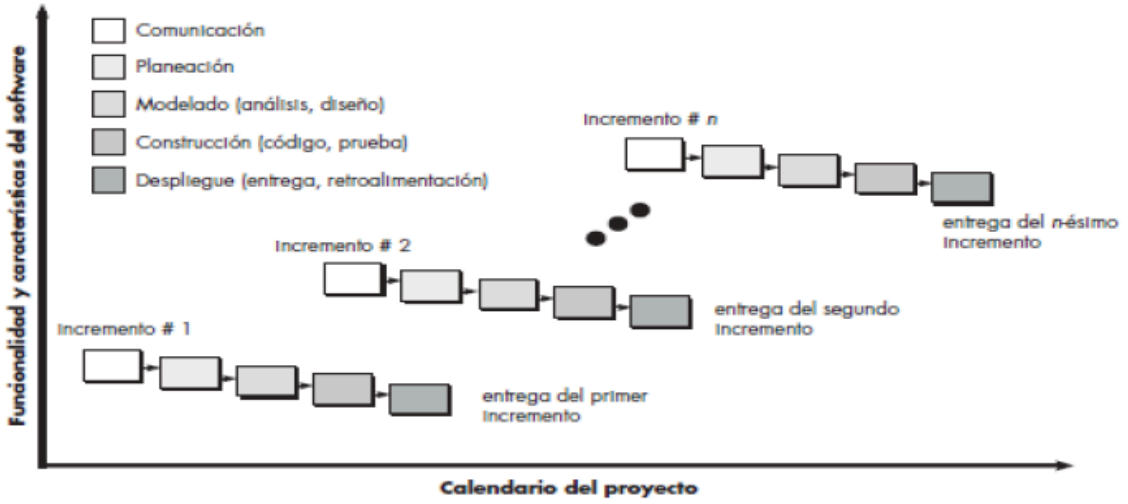
Ciclo de vida de un proyecto de software: Es una representación de un proceso. Gráfica una descripción del proceso desde una perspectiva particular.
Ciclo de vida de un producto de software: Termina cuando genera un producto, cuyo ciclo de vida no va a terminar, sino que va a mantenerse mientras el objeto exista. Lo que quiere decir esto es que el producto necesitará mantenimiento una vez "terminó" de desarrollarse, y se mantendrá bajo este proceso de mantenimiento hasta que sea desechado, donde podremos decir que su ciclo de vida llega al final.

Clasificación de los ciclos de vida

Existen tres ciclos de vida básicos:
1) Modelo secuencial: El proyecto progresa a través de una secuencia ordenada de pasos (fases) y una actividad no puede iniciar sin que la precedente haya sido finalizada, se finaliza tras una revisión al final de cada una de estas. Dirigido por documentos.

Se debe comprender bien los requerimientos para no volver a esta etapa. Algunas pueden llegar a devolver información (retroalimentación). Ejemplo: Ciclos de vida en Cascada. Tienen **dificultades** ya que los proyectos raramente siguen un flujo secuencial, además el cliente obtiene una versión funcional del producto en etapas muy avanzadas del proyecto y encontrar un defecto implica un gran rediseño en la solución

2) Modelo iterativo/incremental: Realiza iteraciones para ganar información, la cual se convertirá en un incremento que posteriormente será incorporado en el sistema. Utilizado en metodologías ágiles. Muy útil para sistemas de requerimientos cambiantes. El cliente es parte del proceso de desarrollo. Se reduce el costo de adaptar los requerimientos cambiantes del cliente. Tienen dificultades ya que se invisibiliza el proceso, genera alto costo de documentar y la estructura del software tiende a degradarse frente a una cantidad considerable de incrementos.



3) Modelo Recursivo: Utilizado en casos particulares (proyectos de alto riesgo). Basado en tomar una característica específica y en una iteración centrarse en esa funcionalidad, en la siguiente la otra, y así sucesivamente. Para llevarlo adelante, se subdivide el proyecto en varios mini proyectos, intentando resolver en cada uno los riesgos más relevantes hasta que no quede ninguno. Actualmente en desuso por ser inútiles y poco funcionales. **Ejemplos:** Ciclo de vida espiral, ciclo de vida en B.

La elección de un ciclo de vida afecta de forma directa a la administración que se debe realizar sobre el proyecto, ya que cada uno de los modelos de proceso poseen características y enfoques distintos. Por ejemplo, si se realiza un análisis de riesgo y se determina que los requerimientos pueden variar mucho a lo largo del desarrollo, se debería elegir un ciclo de vida en el cual el cambio sea permisible, como los iterativos. En este caso, la administración y la planificación del proyecto se ve afectada en la elección del ciclo de vida. Además, durante el desarrollo del proyecto, la gestión de este se realiza en consecuencia de esta elección. “El proceso es una implementación del ciclo de vida que tiene un objetivo que lo guía, que es la creación de un producto.”

Ventajas y desventajas de c/u de los ciclos de vida

Ciclo de Vida	Procesos de desarrollo que lo admite	Características que lo hacen elegible
Secuencial	Definidos	• Alta certeza (baja incertidumbre). • La organización tiene una forma tradicional de trabajar. • No se pueden realizar entregas parciales del software (se debe entregar todo junto). • Eliminar riesgos de planificación.
Iterativo/incremental	Definidos o Empíricos	• Existe incertidumbre. • Volatilidad de requerimientos. • Posibilidad de entregas parciales. • Uso en etapas tempranas del producto.
Recursivo	Definidos	• Mucho control de riesgos en cada iteración. • No se pueden realizar entregas parciales.

Criterios para la selección de un modelo de proceso

Es necesario evaluar los niveles de riesgo, la claridad y cantidad de requerimientos, los conocimientos técnicos con los que cuenta el equipo de desarrollo, los plazos que se tienen para ejecutar el proyecto y la posibilidad de poder validar los resultados del trabajo con el cliente, también el trabajo en equipo.

1. Si los requerimientos son claros y exhaustivos, en un contexto de baja incertidumbre en el que el cliente no requiere obtener el producto rápidamente: Secuencial.
2. Si la situación anterior no se presenta y el cliente desea obtener resultados lo antes posible, con las funciones principales del sistema: Modelo iterativo- incremental.
3. Si el objetivo es disminuir los riesgos presentes en el desarrollo de sistemas grandes y complejos, entonces el modelo recursivo puede ser una buena opción

Ejemplos de modelos de ciclo de vida (11)

1) Code and Fix: Se desarrolla sin especificaciones o diseño y se lo modifica hasta que el cliente esté satisfecho. Los cambios se realizan durante el mantenimiento, lo que es muy caro. No se dedica tiempo a planificar, documentar, hacer control de calidad o cualquiera actividad que no sea codificación pura. Se salta directamente desde un principio a la codificación y los progresos se ven inmediatamente. Requiere poca experiencia, cualquiera que sepa un lenguaje de programación puede familiarizarse con este modelo. Puede ser útil para proyectos pequeños que se tiene la intención de desecharlos después de construirlos.

2) Modelo en cascada puro: Se sigue una secuencia de pasos ordenada y se realiza revisión al final de cada fase. Es conducida por la documentación con fases que no se superponen. Permite encontrar errores en etapas tempranas. Hay exceso de documentación y falta de resultados hasta el final. Se utilizan cuando la definición del producto es estable o los requerimientos de calidad dominan a los costos y tiempos. Funciona bien cuando se tiene un equipo técnicamente débil o con poca experiencia porque provee al proyecto una estructura que ayuda a minimizar el esfuerzo desperdiciado.

3) Modelo en cascada con fases solapadas: Hay un fuerte grado de solapamiento. La documentación es menor, pero es más difícil hacer el seguimiento de progreso.

4) Modelo de entrega por etapas: Se ven signos tangibles de progreso. Es útil cuando el cliente necesita funcionalidad de inmediato y las necesidades se modifican. Debe haber una planificación rigurosa para que funcione.

5) Modelo en cascada con retroalimentación: Es una variación del modelo clásico en que es posible volver a una etapa anterior antes de llegar al final, aunque es muy caro hacerlo.

6) Modelo en cascada con subproyectos: Es un modelo secuencial. Se avanza en cascada hasta el diseño arquitectónico, de ahí en más se avanza en partes dividiendo en subproyectos.

7) Modelo en espiral: Busca minimizar los riesgos haciendo un análisis de riesgos y buscando alternativas al iniciar cada fase. Al final de cada fase se planifica la siguiente y se evalúa si se sigue adelante, evaluando la factibilidad. Este modelo parte el proyecto en mini proyectos que permite manejar mejor los riesgos. En proyectos grandes la identificación de los riesgos puede costar más que el desarrollo. Es un modelo adecuado para proyectos de vida largos, que puede utilizar equipos diferentes en cada ciclo y muestra el progreso al final de este. Por lo general a medida que el costo incrementa, el riesgo disminuye. Mientras más tiempo y dinero gastas, menos riesgos tomás, que es lo que se quiere en un proyecto de desarrollo rápido.

8) Modelo evolutivo: Se fija el tiempo o el alcance mientras el otro se va modificando. Es útil cuando los requerimientos no son claros y se realizan entregas tempranas al cliente. El problema es que es un modelo recursivo que repite todas las tareas y depende de que los diseñadores desarrollen un sistema fácil de modificar.

9) Diseño para cronograma: No se asegura alcanzar la versión final, aunque si una entrega del producto para la fecha definida con los aspectos más importantes. Es útil cuando no hay seguridad sobre las capacidades de programación.

10) Diseño para herramientas: Consiste en incluir solo la funcionalidad soportada por las herramientas de software existentes. Se usa en proyectos muy sensibles al tiempo. Puede combinarse con otros ciclos de vida, pero no se podrán implementar todos los requerimientos.

11) Prototipación evolutiva: Se da cuando hay cambios rápidos de requerimientos y los clientes no se comprometen con los requisitos. Requiere menos documentación, pero no se pueden programar los release. Se desarrolla el concepto del sistema a medida que se avanza. Es útil cuando los desarrolladores no están seguros de que algoritmos o arquitectura óptima utilizar.

Ciclos de vida (Modelos de Proceso) y Procesos de Desarrollo de Software

Proceso de desarrollo de software: Un **proceso** es una secuencia de pasos ejecutados para un propósito dado (IEEE), mientras que un **proceso de software** se define como un conjunto de actividades, métodos, prácticas y transformaciones que la gente usa para desarrollar o mantener software y sus productos asociados.
Las actividades del proceso de desarrollo son la especificación de requerimientos, el análisis, el diseño, la implementación y la prueba. El orden y la relación entre las diferentes actividades está determinada por la elección del ciclo de vida



Las actividades varían dependiendo de la organización y el tipo de sistema a desarrollarse, por lo que deberá ser explícitamente modelado si va a ser administrado.
Se tienen **tres factores** determinantes del proceso. Esto conforma un círculo:

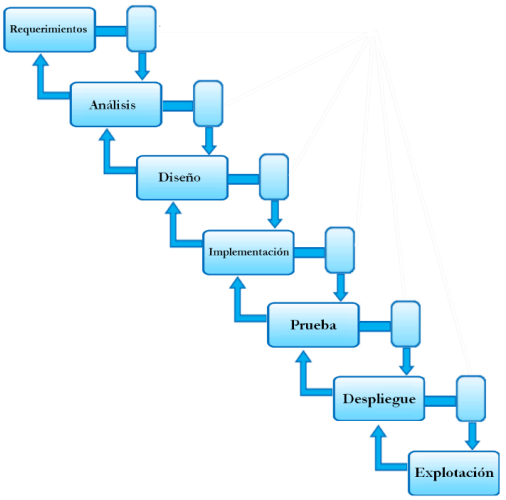
- **Procedimientos y Métodos:** La adaptación de los procesos establecidos para su implementación, dependiendo del contexto, es esencial para la calidad resultante. Definir las actividades que se harán y cuáles no. Además, es relevante que esta información esté documentada para lograr transparencia. Es decir, tener definidos y escritos los procedimientos.
- **Personas motivadas, capacitadas y con habilidades:** Es factor primordial para obtener la calidad del producto del proceso, ya que éste se determina del esfuerzo de las personas. Sin ellas, no se puede lograr construir ningún producto. Por ello, deben estar capacitadas y con habilidades para realizar sus tareas asignadas de la manera correcta y motivados, para lograr aún mayor eficiencia. Recordar que el software es una actividad humano-intensiva.
- **Herramientas y Equipos:** Materiales necesarios para llevar a cabo el proceso que nos permiten que las entradas se transformen en salidas. Se recomienda automatizar la mayor cantidad de actividades posibles, lo que mejora la eficiencia de las personas y puedan concentrarse en tareas que requieran de su capacidad

No hay un proceso ideal, varía en cada organización. Para algunos sistemas, como los sistemas críticos, se requiere de un proceso de desarrollo muy estructurado (proceso definido). Para los sistemas empresariales, con requerimientos rápidamente cambiantes, es probable que sea más efectivo un proceso menos formal y flexible (filosofía ágil).

Procesos de Desarrollo Empíricos vs. Definidos.

Procesos definidos: Son considerados deterministas, ya que ante la misma entrada se pretende que se obtendrá la misma salida. Asumen que, si se aplican una y otra vez, se obtendrán siempre los mismos resultados, a pesar de cambiar de equipo.

- Están inspirados en las líneas de producción.
- La administración y el control provienen de la predictibilidad del proceso. Es decir, el objetivo es lograr una repetibilidad, para poder obtener una previsibilidad de esfuerzo, riesgos, costos, etc. Asociados al desarrollo de software.
- Puedo usar cualquiera de los ciclos de vida.



En realidad, no es posible obtener el mismo resultado ante mismas entradas ni con el mismo equipo, ya que depende sumamente del contexto, momento y las personas, por más parecido sean estos factores. La administración y control del proceso, en muchos de los casos, toman más importancia que el avance del software en sí (más importante la documentación del avance, que el avance en sí).
Nota: Los cuadrados sin texto serían entregables / aprobaciones ->
Ejemplos: PUD (Proceso Unificado de Desarrollo), RUP (Rational Unified Process) ya que ambos de esta forma, desagregan un conjunto de actividades y se define de forma explícita cuáles son las tareas, quién las realizará y en qué momento, a su vez, cuáles serán las entradas y salidas de cada tarea y los artefactos que se generarán.

Procesos empíricos: Se conforman en base a la experiencia interna y externa de las personas que intervienen en un contexto particular. NO es considerado determinista. Dado que el factor diferencial es la experiencia sensible, en estos procesos se pueden obtener diferentes resultados dependiendo del contexto en el cual se apliquen.

- Se basan en ciclos cortos de inspección y adaptación (retroalimentación), porque ante un análisis, se ajustan de mejor forma en aquellos dominios complejos donde prima la creatividad y/o complejidad.
- La administración y el control es por medio de inspecciones frecuentes y adaptaciones para lograr buenas prácticas.
- No se pueden combinar con cualquiera de los tipos de ciclos de vida vistos, se suele recomendar que sea el modelo iterativo-incremental, y el que se suele prohibir es el modelo secuencial debido a que no es viable después de 2 años (lo dice una teoría).
- Asume procesos complicados con variables cambiantes.
- Trabaja bien con procesos creativos y complejos.
- Ejemplo: Scrum. Seguidos por Lean, Agile, Kant.

El empirismo tiene 3 pilares:

- **Transparencia:** Se refiere a la comunicación abierta y honesta de la información relevante a todas las partes interesadas. Promueve la confianza y la colaboración entre los miembros del equipo y las partes interesadas, lo que a su vez conduce a una toma de decisiones más informada y eficaz.
- **Adaptación:** Es la capacidad de adaptarse y ajustar el trabajo y el proceso para hacer frente a las desviaciones y problemas detectados durante la inspección.

Permite una respuesta rápida y efectiva a los cambios y permite mantener la entrega de valor al cliente.

- **Inspección:** Se refiere a la revisión regular y sistemática del progreso del trabajo y los productos resultantes para detectar problemas y desviaciones del plan. Permite una evaluación temprana y continua del progreso y ayuda a tomar decisiones informadas sobre los próximos pasos.

La experiencia que se obtiene en cada uno de los procesos no es extrapolable, es decir, que solo funciona en ese mismo proyecto, y se necesita adaptar los procesos a cada contexto en especial. Los conocimientos obtenidos en el proceso se deben hacer explícitos al grupo de desarrollo, y convertir las habilidades individuales en buenas prácticas del trabajo en grupo (compartir conocimiento). De ahí también sale la necesidad e importancia de las métricas.
Patrón de conocimiento en procesos empíricos: Ciclo donde se parte de una hipótesis (ASUMIR), luego se comienza con la construcción (CONSTRUIR) y en base a algunos puntos que se puedan inspeccionar y analizar se toman mediciones, donde sacamos información y haciendo retrospectiva (RETROALIMENTAR) revisamos (REVISAR) el resultado de la construcción y contrastando la hipótesis inicial con el resultado del proceso, podemos ir adaptando el proceso y seguir construyéndolo (ADAPTAR).
Está basado en una concepción que necesita repetición, por lo que dentro de los procesos empíricos se da uso a un ciclo de vida específico (iterativo e incremental).

Dentro de los procesos definidos podemos encontrar cualquier ciclo de vida, mientras que en los procesos empíricos funcionan con el ciclo de vida iterativo e incremental, lo cual se da ya que les permite adaptar y mejorar el proceso, a su vez en cada iteración poder retroalimentarse y generando la experiencia que requieren para realizar la adaptación.

Componentes de un Proyecto de Sistemas de Información (PPT 6)

Proyecto

Un proyecto de software es un esfuerzo temporal que requiere del acuerdo de un conjunto de especialidades y recursos para la creación de un producto, servicio o resultado único. Es una unidad organizativa para adaptar el marco teórico del proceso, dependiendo de las personas y recursos con los que se cuente.
Define el proceso y tareas que se van a realizar, el personal que se encargará de las actividades y los mecanismos que se implementarán para valorar riesgos, controlar el cambio y evaluar la calidad. Analizaremos sus características:

- **Únicos:** El resultado de un proyecto será único e irrepetible para cualquier otro proyecto, por más parecido o igual sean sus elementos componentes.
- **Orientado a objetivos:** Estos objetivos deben ser no ambiguos, medibles y alcanzables. Deben guíar al proyecto.
- **Duración limitada de tiempo:** El proyecto tiene un principio y un fin.
- **Conjunto de tareas interrelacionadas basadas en esfuerzo y recursos:** Se definen las tareas, sus dependencias, los recursos y personas asignadas, permitiendo manejar la complejidad inherente de los sistemas.

Administración de proyecto de software

Es la aplicación de conocimientos, habilidades, herramientas y técnicas a las actividades del proyecto para satisfacer los requerimientos de este y poder obtener como resultado el producto esperado. Se divide en dos grandes actividades:

- **Planificación:** Incluye identificar los requerimientos, establecer objetivos claros y alcanzables, y adaptar las especificaciones, planes y el enfoque a los diferentes intereses de los involucrados (stakeholders).
- **Monitoreo y control**

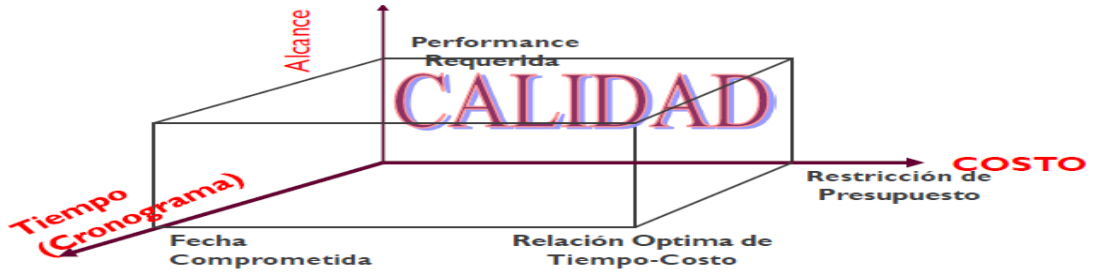
Una buena gestión no puede garantizar el éxito del proyecto. Sin embargo, una mala gestión usualmente implica una falla en el proyecto. Un proyecto planificado puede fracasar también, lo importante de la planificación es el acto de planificar, no el resultado. El acto de ponerse en el ejercicio de pensar tanto: riesgos, objetivos, alcances, estimaciones, etc.
Las **metas** más importantes de la administración de proyectos son:

- Entregar software al cliente en el tiempo acordado.
- Mantener los costos dentro del presupuesto general.
- Entregar software que cumpla con las expectativas del cliente.
- Mantener un equipo de desarrollo óptimo y con buen funcionamiento.

Atributos particulares o complejidades en la administración de proyectos de software:

- **El producto es intangible:** No poder ver ni tocar el producto que se obtiene al ejecutar el proyecto dificulta la medición del progreso del proyecto.
- **Los proyectos de software suelen ser excepcionales:** Los conocimientos aprendidos en un proyecto de software son difícilmente extrapolables a otro proyecto. Causa = factores tecnológicos.
- **Los procesos de desarrollo son variables y específicos de la organización:** A pesar del intento de estandarizar los procesos de desarrollo, la realidad es que no es posible predecir de manera confiable qué proceso es el adecuado para el proyecto

Triple restricción



Las variables de restricción de un proyecto son 3: **tiempo, costo y alcance.** El balance de estos tres factores afecta directamente la **calidad** del proyecto, ya que un proyecto de alta calidad es aquel que entrega el producto, servicio o resultado requerido, satisfaciendo los objetivos en el tiempo estipulado y con el presupuesto planificado. Es responsabilidad del Líder de proyecto balancear estas variables. El concepto de la triple restricción hace referencia a que no es posible fijar de forma arbitraria a las 3 variables, sino que será posible fijar 2 y la tercera se ajustará a lo definido. No se negocia la calidad, si no puedo modificar tiempo y/o costo, debido a que es como entregar un software sin probarlo. La calidad del producto que le entrego al cliente se ve afectada cuando no puedo equilibrar.

- **Alcance (¿Qué trata de alcanzar?):** Son los requerimientos que se incluyen en el proyecto y definen la funcionalidad que tendrá el producto a entregar al cliente. En las metodologías tradicionales el alcance es lo primero que se determina y se deja fijo en función de las necesidades del cliente, mientras que en ágil se va definiendo, refinando y adaptando en base a las retroalimentaciones del cliente. Esta variable es la que primero se negocia con el cliente.
- **Tiempo (¿Cuánto tiempo me llevaría?):** Define la fecha de calendario en la cual se compromete a finalizar el proyecto y entregarle el producto resultante al cliente. En las metodologías ágiles, el tiempo es fijo.
- **Costo (¿Cuánto debería costar?):** Define el costo de todos los recursos implicados en el desarrollo del proyecto. Esto incluye equipamiento, infraestructura, equipos, salarios, entre otros.

En la realidad, el costo y el tiempo del proyecto varían de forma directa con la definición del alcance del producto, es decir, el alcance es directamente proporcional al tiempo y al costo. Cuando el alcance aumenta (mayor cantidad de funcionalidades), el tiempo y dinero (costos) necesarios también deben aumentar, para lograr abordar un proyecto más grande.

El problema en la aplicación de esta visión en enfoques tradicionales es que, al utilizar procesos definidos con ciclos de vida en cascada, los requerimientos se definen al comienzo del proyecto, por lo que cualquier modificación o cambio en el alcance del producto final, modifica el tiempo y presupuesto de este. Estos cambios (frecuentes, si no genera necesidad de cambios es porque no lo usa nadie al sw), generan una baja en la calidad del producto final ya que, que se sacrifican funcionalidades o bien, se saltean etapas de testing, etc. debido a que el cliente siempre espera que se entregue lo pactado, respetando el presupuesto y tiempo de entrega. El producto de software se desarrolla incrementalmente en una serie de pasos en cada una de sus nuevas versiones.

Líder de proyecto

Es el encargado de todas las tareas de gestión que hacen que el equipo de proyecto sea guiado a lograr el objetivo. Es el intermediario entre el ente que regula el avance del proyecto y demás involucrados con el equipo. A su vez también se encarga de administrar todos los recursos, organizar el trabajo de las personas y hacer el seguimiento de la planificación verificando lo planeado. Además de estimar y planificar.

Rol: En un enfoque tradicional, el líder de proyecto realiza la toma de decisiones en su totalidad sin la inclusión de la opinión del equipo. El líder asigna las tareas que deben realizar los integrantes del equipo, y además, maneja todas las relaciones con todos los stakeholders del proyecto (clientes, gerencias, contratistas, stakeholders en general). En procesos empíricos, no es necesario un PM ya que los proyectos se autogestionan.

Un líder debe tener habilidades necesarias para tratar con la gente, como habilidades de motivación, organización, innovación, empatía, comunicación, liderazgo.
Además también con habilidades duras, relacionadas a los conocimientos del producto, técnicas y herramientas. Enfoque basado en la resolución de problemas, identidad administrativa, logro con recompensas e influencia

En las nuevas metodologías, sus responsabilidades son:

- Definir el alcance del proyecto.
- Identificar a los involucrados, recursos y presupuesto.
- Detallar las tareas, estimar los tiempos y requerimientos.
- Identificar y evaluar riesgos.
- Preparar planes de contingencia.
- Controlar hitos y participar en las revisiones de las fases del proyecto.
- Administrar el proceso de control de cambios.
- Producir reportes de estado.

Equipo de proyecto

Es un grupo de personas comprometidas con alcanzar un conjunto de objetivos de los cuales se sienten mutuamente responsables. Características:

- Cuentan con conocimientos, habilidades técnicas, motivación, experiencias y diversas personalidades. Mejora continua y aprendizaje mutuo.
- Usualmente son un grupo pequeño, esto permite lograr una comunicación efectiva.
- Trabajan en forma conjunto con espíritu de grupo, convirtiéndose en un equipo cohesivo donde prima la sinergia de este. Para eso, se debe lograr un equipo maduro y estable, no se agrega o cambia un integrante constantemente.
- Sentido de responsabilidad como una unidad y se focalizan en el cumplimiento de las metas grupales.

En equipos cohesivos:

- El grupo establece sus propios estándares de calidad.
- Los individuos aprenden de los demás y se apoyan mutuamente.
- El conocimiento se comparte.
- Prevalece la mejora continua.

Stakeholders

Un stakeholder es una persona, grupo u organización que tiene interés en un proyecto o puede verse afectado por él, o influir en su desarrollo y resultados.

- **Ejemplos:** Clientes, usuarios, desarrolladores, gerentes, proveedores, organismos reguladores y propietarios del producto.

Plan de proyecto y sus 8 actividades

El plan de proyecto, (o también llamado plan de desarrollo de software en el ámbito del software), es un artefacto de gestión que cumple la función de “hoja de ruta” de un proyecto, en la cual se documentan todas las decisiones que se toman a lo largo del desarrollo; en términos de: el proceso, la división de tareas a realizar, asignación de responsabilidades, equipos de trabajo, calendario de desarrollo, actividades de soporte (planes de contingencia, mecanismos para control de cambios, evaluar calidad y valorar riesgos). Brindando respuesta a todas esas preguntas se logra un plan detallado que servirá como guía del proyecto. Mediante este plan de proyecto documentamos:

- **¿Qué es lo que hacemos?:** El alcance del proyecto.
- **¿Cuándo lo hacemos?:** Calendarizamos.

- **¿Cómo lo hacemos?:** Recursos y decisiones disponibles.
- **¿Quién lo va a hacer?:** Asignación de las tareas.

Responder a estas preguntas, implica las siguientes 8 actividades:
1) Definición del alcance del proyecto. 2) Definición de proceso y ciclo de vida. 3) Estimación. 4) Gestión de riesgos. 5) Asignación de recursos. 6) Programación de proyectos. 7) Definición de métricas. 8) Definición de controles.

La idea primordial del plan de proyecto es que sea algo vivo a lo largo del proyecto, donde se retroalimente y se corrija a medida que el proyecto avanza. Siempre será útil si está en permanente actualización, ya que pueden surgir variaciones no estimadas en el proyecto.

1) Definición del alcance del proyecto.

Alcance del proyecto	Alcance del producto
Es todo el trabajo y solo el trabajo que debe hacerse para entregar el producto o servicio con todas las características y funciones especificadas. El cumplimiento del alcance del proyecto se mide contra el plan de proyecto.	Son todas las características que pueden incluirse en un producto/servicio. El cumplimiento del alcance de producto se mide contra la Especificación de Requerimientos (ERS) que contiene los requerimientos funcionales y no funcionales.

Relación: Si el producto a construir es grande, las tareas a definir en el proyecto van a ser más o van a requerir más tiempo y recursos, por lo que para definir el alcance del proyecto se debe primero definir el alcance del producto. Es importante aclarar que el alcance del proyecto es menor que el alcance del producto.

2) Definición de proceso y ciclo de vida

Lo importante es que al iniciar un proyecto se deberá definir qué proceso quiero utilizar y qué ciclo de vida voy a utilizar.
El proceso de desarrollo es el conjunto de actividades que necesito ejecutar para construir el producto de software (Ej. PUD) y el ciclo de vida es de qué manera ejecuto esas actividades. En la gestión tradicional y los procesos definidos se puede elegir el proceso y cualquier ciclo de vida a utilizarse para llevar a cabo esté, mientras que en la gestión ágil el único ciclo de vida que se puede utilizar es el iterativo.

3) Estimación

En un proyecto de desarrollo de software, se trata de estimar para predecir el valor de un elemento relacionado con el proyecto o con el producto. Se debe estimar todo al momento de realizar un plan de proyecto, ya que no hay certezas de nada. Donde deberemos estimar todas las características necesarias para la planificación, lo cual se hará con un cierto nivel de incertidumbre. En la metodología tradicional, existe un orden definido y recomendado:

1. **Tamaño:** Definir el producto a construir. Para estimar el tamaño de un proyecto se utilizan los requerimientos tomados en la etapa de requisitos. Una desventaja de esta metodología tradicional es no responder a cambios de forma flexible. El tamaño de un producto se puede estimar en muchas escalas, por ejemplo: líneas de código, cantidad de requerimientos, puntos de función ajustados, etc.

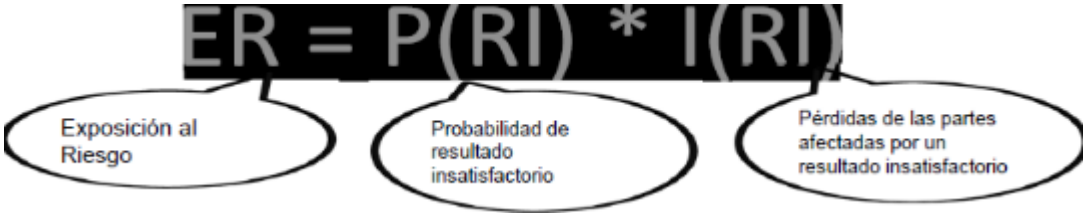
2. **Esfuerzo:** Refiere a la cantidad de horas persona lineales haciendo una actividad por vez, dentro del desarrollo del producto. Cada rol, puede tener un valor de hora distinto o bien, se puede realizar una estimación en horas planas, en donde no se hace distinción de valor por rol. El esfuerzo es el tópico que más repercute en el costo del proyecto, abarcando casi el 80% de estos. No contempla tiempo para comer/ir al baño etc.
3. **Calendario:** Permite responder al cuándo se van a realizar las tareas definidas. No es necesario realizar una calendarización muy específica a nivel de día, sino que se escala a tareas a realizar en, por ejemplo, un mes del año o incluso un trimestre.
4. **Costo:** Determinar un presupuesto necesario – Usualmente definido al final ya que debe ajustarse a la realidad
5. **Recursos críticos:** Tanto humanos como físicos que nos van a hacer falta.

4) Gestión de riesgos

Riesgo es la probabilidad de ocurrencia de una pérdida o daño que afecte de forma directa al proyecto y pueda comprometer el incumplimiento de su objetivo.
Es de suma importancia identificar y definir los riesgos a los que está atado un proyecto, para así poder analizarlos, y realizar una cuantificación de estos, teniendo en cuenta la exposición del proyecto ante los mismos, para así definir a cuáles se debe atender y sobre cuáles se debe hacer un seguimiento.
Podemos clasificarlos en:

- **Riesgos del proyecto:** Aquellos que alteran el calendario o los recursos del proyecto. Un ejemplo sería la renuncia de un diseñador experimentado.
- **Riesgos del producto:** Aquellos que afectan la calidad o el rendimiento del software a desarrollar. Un ejemplo es la falla que presenta un componente que se adquirió al no desempeñarse como se esperaba.
- **Riesgos organizacionales o de negocio:** Aquellos que afectan a la organización que desarrolla o que adquiere el software.

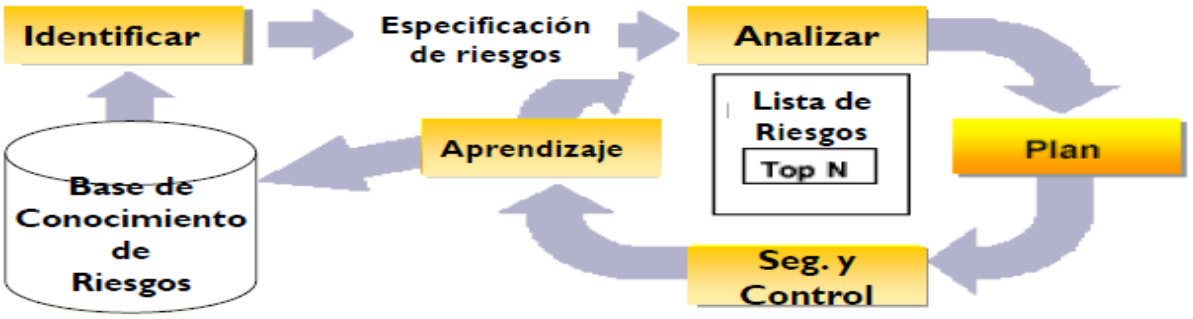
Para medir los riesgos se utiliza lo que se conoce como exposición al riesgo (probabilidad x impacto), la cual se obtiene como el producto entre la probabilidad de ocurrencia de la amenaza por el impacto que generaría si realmente ocurriera. Este valor permite organizar y gestionar los riesgos, y se debe hacer foco en reducir la exposición del riesgo.



Antes los riesgos identificados, es posible tomar 3 actitudes:

- **Negación:** Hacer de cuenta que los riesgos no existen.
- **Gestión Reactiva:** Esperar a que el riesgo ocurra y recién ahí ver que hacer.
- **Gestión Proactiva:** Ir trabajando sobre el riesgo generando planes de mitigación y de contingencia

Gestión del riesgo: Se analiza la probabilidad de ocurrencia y del impacto.



5) Asignación de recursos / responsabilidades

La asignación de responsabilidades a personas permite asignar roles a los integrantes del equipo de trabajo. En el entorno de procesos definidos (PUD, por ejemplo), estos roles se ven definidos en el concepto de trabajadores.
Se debe hacer mucho foco en la selección del personal de un proyecto, ya que las capacidades de los integrantes del equipo afectan a la calidad del software y al correcto desarrollo del proyecto en términos de tiempo, por ejemplo, ya que el desarrollo de software es una actividad HUMANO-INTENSIVA.

6) Programación de proyectos (calendarización)

En esta etapa se trata de definir en detalle, cada tarea que debe ser cumplida en el desarrollo. Cuando se habla de detallar las tareas, se hace referencia a descomponer el proceso de desarrollo en actividades refinadas a nivel de días, identificando quien la realiza, cuando debe realizarse y cuánto esfuerzo debe llevar (teórico). EJ: Diagrama de Gantt.

7) Definición de métricas

Las métricas se definen como una medida numérica que aporta visibilidad sobre el avance o estado del proyecto, proceso o producto en un momento determinado del desarrollo. Al realizar la planificación del proyecto, es necesario definir de forma clara y no ambigua, cada una de las métricas asociadas a cada dominio (producto, proyecto o proceso) en conjunto con la forma de calcularlas. Además, es imprescindible que las métricas sean representativas. Esto se debe a que para planificar nos basamos en estimaciones, estas métricas permitirán saber si se cumple lo planificado o no.
Las métricas del proyecto se consolidan para crear métricas de proceso que sean públicas para toda la organización del software.
A diferencia de las metodologías ágiles, acá se definen métricas que miden y exponen los avances. En cambio en ágil, el mayor indicador de avance de un producto, son los incrementos que se entregan en cada iteración.
Las métricas básicas para un proyecto de software se clasifican en tamaño del producto, esfuerzo, tiempo (calendario) y defectos
Estas métricas hay que mantenerlas simples, las preguntas a hacerse son:

- ¿Nos da más información que la que tenemos ahora?
- ¿Es esta información de beneficio práctico?
- ¿Nos dice lo que queremos saber?

Además hay que balancear en el triángulo de recursos, alcance y tiempo, con las restricciones del triángulo de tecnología, personas y proceso.

Forma ideal:

Desarrollador

1. Esfuerzo
2. Esfuerzo y duración estimada y actual de una tarea.
3. % de cobertura por el unit test
4. Numero y tipo de defectos encontrados en el unit test.
5. Numero y tipo de defectos encontrados en revisión por pares.

Organización

1. Tiempo Calendario
2. Performance actual y planificada de esfuerzo.
3. Performance actual y planificada de presupuesto
4. Precisión de estimaciones en Schedule y esfuerzo
5. Defectos en Release

Equipo de Desarrollo

1. Tamaño del producto
2. Duración estimada y actual entre los hitos más importantes.
3. Niveles de staffing actuales y estimados.
4. Nro. de tareas planificadas y completadas.
5. Distribución del esfuerzo
6. Status de requerimientos.
7. Volatilidad de requerimientos.
8. Nro. de defectos encontrados en la integración y prueba de sistemas.
9. Nro. de defectos encontrados en peer reviews.
10. Status de distribución de defectos.
11. % de test ejecutados

8) Definición de controles

En la planificación de los controles del desarrollo, se toma como base las métricas ya definidas, y verifica que todo se está haciendo en concordancia con lo establecido. En esta planificación se definen reuniones periódicas, reportes o informes necesarios, etc.

Factores para el éxito o fracaso

Factores para el éxito	Causas de fracasos
• **Monitoreo y feedback:** Tener monitoreo permanente y generar acciones correctivas cuando sea necesario para evitar una desviación mayor. • **Misión/objetivo claro:** Saber hacia dónde vamos. • **Comunicación:** En todos sus aspectos, con el líder del proyecto, con el equipo, con clientes y stakeholders.	• Fallas al definir el problema. • Planificar basado en datos insuficientes. • La planificación la hizo el grupo de planificaciones. • No hay seguimiento del plan de proyecto. • Plan de proyecto pobre en detalles. • Planificación de recursos inadecuada. • Las estimaciones se basaron en “supuestos” sin consultar datos históricos. • Nadie estaba a cargo. • Mala comunicación.

Vinculo proceso-proyecto-producto en la gestión de un proyecto de desarrollo de software.



La disciplina de Ing. En software se mueve básicamente en tres dimensiones (proceso, proyecto y producto, 3P), pero debemos considerar a las personas como la cuarta dimensión (y cuarta P), ya que son muy importantes y un factor clave para el éxito.

- **Personas:** Son todas aquellas involucradas en el proyecto. Es lo más importante e involucra a todos los stakeholders.
- **Proceso:** Es el conjunto de actividades, es decir que tenemos que hacer, para hacer software (Requerimientos), dentro de este proceso encontramos los anteriormente mencionados como empíricos y definidos. Es abarcativo o al menos esa es la idea, después lo adaptamos al proyecto en particular
- **Proyecto:** Es una unidad de trabajo/gestión del trabajo, siendo el medio por el cual se organiza el trabajo, recursos y personas. Utiliza los ciclos de vida, en donde se indicará como llevar a cabo el proceso elegido. Una vez finalizado el ciclo de vida del proyecto se obtiene como resultado el producto. Para ello deberá cumplir ciertas características. En la carrera, siempre trabajamos enfocados en la creación de producto, ahora nos vamos a enfocar en proyecto.
- **Producto:** Son versiones de software generadas para entregarle al cliente. Trasciende y tiene más vida que el proyecto (que tiene fecha de inicio y fecha de finalización)

Se dice que el proceso, automatizado con herramientas, se adapta al proyecto, al cual se incorporan personas, y de este se obtiene como resultado un producto.

En el enfoque tradicional, previo a la definición del proyecto, es necesario determinar los objetivos y el ámbito del producto para de esta manera poder realizar las estimaciones pertinentes. Los objetivos identifican las metas globales del producto sin especificar cómo se van a lograr. El ámbito del producto define las funciones y comportamientos que caracterizan al producto que utilizara cliente.
Para obtener como resultado un producto o servicio (ya sea SaaP o SaaS), es necesario que el proyecto adopte un proceso de desarrollo para poder definir qué actividades, métodos, prácticas y transformaciones se utilizarán, mediante las cuales se va a obtener el software. Dentro de cada proyecto de desarrollo de software, es necesario utilizar un proceso diferente dependiendo del tipo de producto que se desea desarrollar, ya que puede que la complejidad de este sea un determinante en la elección del proceso. Es por esto, que es necesario adaptar los procesos a cada uno de los proyectos en los cuales se trabaja, es decir, utilizar solo lo que verdaderamente hace falta para el desarrollo de este, ya que no existe un proceso ideal que sirva para cualquier tipo de proyectos. Es decir, el proceso es una definición teórica de lo que debería hacerse para tomar los requerimientos y transformarlos en un producto o servicio, guiados por el objetivo de lo que se quiera construir.

El **proceso** proporciona un marco conceptual en donde se establece un plan completo para el desarrollo del software. Define cómo se va a desarrollar el software, estableciendo un conjunto de actividades. Las disciplinas de gestión permiten que las actividades del marco conceptual se adapten a las características del proyecto de software y a los requerimientos del equipo. Las disciplinas de soporte son independientes del marco conceptual y recubren al modelo del proceso, es decir que atraviesan todas las actividades del proceso de desarrollo. Se debe definir el modelo de proceso a utilizar: secuencial, iterativo o recursivo.

Luego, el marco conceptual que incluye las actividades fundamentales del desarrollo del software se adapta al modelo elegido.

Un **proyecto** de desarrollo está integrado por un equipo de trabajo, dentro del cual deben estar los roles bien definidos, para que cada uno de los integrantes asuma la responsabilidad que le corresponda, y se tengan en claro los alcances de cada uno de estos roles. El equipo va a basar su trabajo en el proceso adoptado, en el cual se van a tomar sólo aquellas actividades y formas de trabajo que deberán utilizar y seguir para lograr el desarrollo del **producto**/servicio deseado.

Las **personas** son la parte más importante, debido a que el desarrollo de software es una actividad humano-intensiva. Si se adopta el "mejor proceso" para el desarrollo de un software, y además se destina mucho esfuerzo a planificar el proyecto de desarrollo, de nada servirá si no tenemos un equipo capacitado que reúna las habilidades y herramientas necesarias para el desarrollo del mismo.
En el desarrollo de un proyecto, se busca utilizar diversas herramientas que se adapten y permitan automatizar o facilitar las actividades que están definidas en el proceso adoptado. Por ejemplo, el uso de un software de diseño para realizar el análisis y diagramación de clases.

Unidad 1 Anexo: No Silver Bullet

Brooks plantea la pregunta de ¿Por qué desarrollar software es tan difícil?: A lo que adjunta la respuesta de "porque una parte importante de esa dificultad pertenece a la naturaleza misma del software". Cuando se complica el proyecto de software, aparece la esperanza de encontrar una "bala de plata", es decir, una solución tecnológica o metodológica capaz de producir por sí sola una mejora enorme en la productividad y calidad del desarrollo. Brooks sostiene que no existe esa bala de plata.
Las dificultades que plantea son divididas en esenciales o accidentales:

Dificultades esenciales: Es todo aquello que trae aparejado el construir software: conceptos abstractos, estructuras de datos, algoritmos, invocación de funciones, etc. (propios de la naturaleza del sw). La esencia del software resiste la automatización porque posee características únicas en la ingeniería humana. 4 aspectos la caracterizan:

- **Complejidad:** Propiedad inherente, debido a la abstracción del software al crecer el tamaño del software, su complejidad crece de manera exponencial, no lineal. Esto conduce a muchos problemas: dificultad de comunicación entre miembros de equipos, deficiencias en el producto, errores en cumplimiento de fechas, excesos en costos, etc.
- **Conformidad:** El software debe realizar aquello que el cliente necesita, esto es: debe adecuarse a sus requerimientos.
- **Mutabilidad:** El software está sujeto a constantes cambios. A diferencia de objetos tangibles, el software muta y sufre cambios a lo largo del tiempo. En cambio, en muchos objetos esto no es así, ya que se reemplazan por nuevos productos (no tomo el objeto y lo modifico, creo uno nuevo).

- **Invisibilidad:** Al ser algo no visualizable, se dificulta mucho trabajar con representaciones geométricas, como puede ser con estructuras mecánicas. Nos podemos ayudar con grafos.

Dificultades accidentales: Tiene que ver con aquellos aspectos que dificultan la construcción del software, aparecen por cómo desarrollamos o representamos el software, pero han sido solucionados a lo largo del tiempo. Esas soluciones son:

- **Lenguajes de programación de alto nivel:** Permiten abstraer el proceso de construcción del software, independizándose del hardware en el que se ejecutarán. Esto elimina la complejidad propia de una máquina, que nada tiene que ver con el software que se desea construir.
- **Tiempo compartido:** Reducción de los tiempos de respuesta de un software. Esto permite no desviar el foco de lo que se desea construir.
- **Entornos de programación unificados:** Facilitan la construcción del software al integrar librerías, formato de archivos, frameworks, etc.

Resolución de dificultades accidentales:

- **Programación orientada a objetos:** Permite a los diseñadores del software, abstraerse de tecnicismos para expresar su diseño simplemente en un lenguaje de máquina. Esto facilita el diseño, pero sólo elimina una dificultad accidental, no elimina la propia dificultad esencial de diseñar software.
- **Inteligencia Artificial:** Si bien la IA permite resolver muchas cuestiones que reemplazan acciones humanas, siempre tiene que haber un equipo de personas qué programe y “enseñe” a esa IA cómo actuar. Con lo cual, el factor humano no se reemplaza del todo.
- **Sistemas expertos:** Son sistemas que aplican la IA y permiten sugerir diferentes situaciones como estrategias de Testing, optimizaciones de código, sugerencias en nombrado de variables, etc. Igualmente, no se quita el factor humano ya que se necesita encontrar expertos y desarrollar técnicas eficientes para extraer lo que ellos saben para destilarlo dentro de bases de reglas de estos sistemas. El prerrequisito esencial para construir un sistema experto es tener un experto
- **Programación automática:** Esto se planteó como un reemplazo a los programadores, pero en lugar de ser eso, terminan siendo un lenguaje de más alto nivel que el programador debe parametrizar.
- **Programación gráfica:** Permite programar a un nivel más alto de abstracción que la programación tradicional, pero sigue estando el factor humano presente, con lo que solo se logra dar una mayor abstracción al programador sin que este tenga la necesidad de aprender sintaxis del lenguaje.
- **Verificación de programas:** Lo que se logra con la verificación de programa, es establecer que un programa pasa las pruebas matemáticas de verificación que se proveen. No permite verificar que se cumplan todas las necesidades del cliente que fueron previstas en el software en forma de funcionalidades, sino que verifica que las funcionalidades desarrolladas no tengan errores. Además, las pruebas matemáticas pueden también estar sometidas a errores.
- **Environments and tools:** La mayoría de los nuevos entornos inteligentes tan solo nos permiten librarnos de errores semánticos y sintácticos simples.
- **Estaciones de trabajo poderosas:** El incremento de potencia y memoria de las estaciones de trabajo individuales no soluciona el déficit en el desarrollo el software.

La edición de programas y documentos son soportadas totalmente por las máquinas de hoy en día. Una mejora en tiempos de compilación no representa un avance significativo.

Resolución de dificultades esenciales:

- **Comprar, antes que construir:** Explorar en el mercado soluciones que se puedan conseguir para reemplazar lo que se quiera construir. Si existe un mercado que lo ha demandado, comprarlo siempre será más barato que construirlo. Esto se debe a que al tener una gran cantidad de clientes que lo compran, el costo de ese software se amortiza. Mirarlo como que el uso de N copias de software, multiplica la productividad de ese software N veces. Lógicamente este enfoque es difícil de utilizar para necesidades muy puntuales, donde no existe una porción de mercado que lo haya demandado previamente.
- **Usar prototipos y refinar requerimientos:** La parte fundamental de construir software es saber qué se debe construir. La extracción iterativa de requerimientos y su posterior refinamiento permiten tomar las necesidades de los clientes en sucesivas iteraciones, debido a que ellos mismos no saben expresar ni conocer muchas veces cuáles son esas necesidades que tienen. Además, es prácticamente imposible que sepan expresar con detalle y precisión todos sus requerimientos para el software que necesitan. El desarrollo de prototipos rápidos permiten cubrir esa extracción iterativa de requerimientos, al simular interfaces y funciones principales de un software que se necesita construir. Esto permite obtener un feedback valioso del cliente, para que todos comprendan mejor cuáles son las funcionalidades que se necesitan, y cómo se deben ejecutar.
- **Desarrollo incremental: (crecer y no construir software):** Incrementar el software añadiendo funcionalidades mientras estas se prueban, usan y testean.
- **Grandes diseñadores:** La parte más difícil de construir software es decidir qué construir. Por ejemplo el nuevo lenguaje de alto nivel (la IA) para que funcione, el humano debe especificar con precisión matemática las restricciones, estados y lógicas de negocio. Si el humano no comprende la complejidad esencial, el prompt será incompleto. La IA no deduce la intención no escrita. Históricamente la programación automática solo funcionó en dominios con parámetros altamente restringidos. El flujo total del sistema sigue estrangulado por la capacidad humana de diseño conceptual, sin importar cuán rápido se genere el código abajo. El diseño sigue siendo un acto humano. Las diferencias entre diseños buenos y grandes se acercan a una orden de magnitud. Los grandes diseños provienen de grandes diseñadores, no de balas de plata.

Unidad 2: Gestión Lean-Agile de Productos de Software

Filosofía ágil: Movimiento surgido por desarrolladores, donde se acordaron un conjunto de enunciados a los que se llamaron manifiesto ágil. El objetivo primordial es entregar software

de forma frecuente en un entorno cambiante, involucrando al cliente en el proceso de desarrollo, con el sw como una serie de incrementos, con rápida retroalimentación.

Manifiesto: Es un compromiso que hacen todas las personas involucradas en un proyecto para trabajar de una determinada manera independientemente de las prácticas que realice c/u. La forma de trabajo será sustentada en los conceptos de los procesos empíricos. Su objetivo es lograr un equilibrio entre seguir procesos rigurosos y formales en el desarrollo de software y trabajar de manera eficiente y efectiva para entregar un producto de calidad.

Ágil: Forma de llamar a está filosofía. No es una metodología ni un proceso, si no que es una ideología que cuenta con un conjunto definido de principios que guían el desarrollo de un producto. Buscando encontrar un equilibrio entre ningún proceso y demasiado proceso. Los métodos ágiles son adaptables en lugar de predictivos y son orientados a la gente en lugar de orientados al proceso. Es aplicado en un contexto complejo o con incertidumbre.

- **Ejemplo de Frameworks ágiles:** FDD, Cristal, XP, SCRUM, ATDD.

Manifiesto ágil: Es un documento redactado en 2001 por expertos de la Ingeniería en Software, que promueve la filosofía del desarrollo ágil del software. Es un acuerdo con 4 valores y 12 principios.

Requerimientos Ágiles (PPT 3)

4 valores ágiles

1) Individuos e interacciones, por sobre procesos y herramientas: Es preferible un equipo de personas motivadas con un buen funcionamiento y conocimiento, donde la comunicación sea fluida, pero con herramientas pobres para su trabajo, que tener un equipo de individuos desmotivados o poco funcional con las mejores herramientas y procesos. Así lo expresa el manifiesto, debido a que el desarrollo de software es una actividad humano-intensiva, y los procesos ágiles intentan capitalizar las fortalezas de cada individuo (como así también paliar las debilidades individuales), en lugar de intentar homogeneizar un equipo, para así lograr una armonía que permita fortalecer la unión y motivación dentro del equipo. Los procesos y herramientas deben ser un complemento para mejorar la eficiencia y éstos se deben adaptar al equipo y no al revés.

2) Software funcionando por sobre documentación extensiva: Priorizar el software asegura que se tengan versiones estables, incrementales y mejoradas del software al finalizar cada iteración. Esto permite que a medida que el software crezca en cuanto a funcionalidades, tras cada iteración puede ir mostrándose a los usuarios finales y obtener una retroalimentación de ellos, para que el equipo de desarrollo se asegure de estar trabajando siempre en aquellas funcionalidades de mayor valor para el cliente y aquellas que satisfagan sus expectativas. Solo se documenta aquello que agregue valor al producto y esté centrado en el cliente (valor agregado).
Existe la necesidad de mantener información sobre el producto de software y sobre el proyecto que se realiza para obtener un producto y el enfoque ágil lo que plantea es que se debe generar la documentación cuándo sea necesario. No plantea “no generar documentación”. Los documentos no pueden sustituir, ni pueden ofrecer la riqueza y

generación de valor que se logra con la comunicación directa entre las personas y a través de la interacción con el software. Mínimamente documentar la arquitectura por más que no se diga nada de este tema de forma oficial.

3) Colaboración con el cliente por sobre negociación contractual: Enfatizado en la importancia de trabajar en colaboración con el cliente y comprender sus necesidades y requisitos en lugar de simplemente negociar contratos y acuerdos formales.

- **Relación con la triple restricción:** Se da ya que en las negociaciones contractuales el cliente firma un contrato con el equipo definiendo un alcance/costo/tiempo determinado y luego quiere realizar cambios. Estos problemas se evitan al involucrar al cliente de manera más activa en el proyecto y se fomenta una actitud de colaboración e integración. Siendo fundamental que el cliente esté dispuesto a participar y sumarse a este enfoque, en caso de que no, la metodología ágil no se podrá aplicar.

4) Responder a cambios por sobre seguir un plan: Esto se plantea debido a que es imposible que los usuarios sepan con detalle y certeza todas las funcionalidades que necesitan, y a su vez, con el paso del tiempo surgirán nuevas necesidades, las cuales en un primer momento no se habían detectado. Esto está ligado al entorno cambiante en el que se desempeñan los clientes que demandan un software, por lo que un equipo ágil está abierto a los cambios, para poder responder ante ellos de una manera rápida y eficiente entregando el mayor valor posible para el cliente y los usuarios, en lugar de apegarse y seguir un plan definido confeccionado en etapas tempranas de requerimientos.
Los principales valores de la gestión ágil son la inspección y la adaptación, diferentes a los de la gestión de proyectos tradicional: planificación y control que evite desviaciones del plan. (Los cambios son vistos de buena forma).
Analizando en concreto el proceso de desarrollo de un equipo ágil, al finalizar cada iteración, la experiencia de desarrollar el incremento genera cierto conocimiento en el equipo, que afecta a la planificación en los distintos niveles. De forma similar, mostrar o desplegar esos incrementos a los usuarios finales, también generará nuevo conocimiento para esos procesos de planificación.
Cualquiera sea el motivo, este nuevo conocimiento puede inducir a cambios, los cuales el equipo ágil deberá incluir en su planificación, de forma que siempre se apunte a entregar un producto con el mayor valor posible. En lugar de una ERS firmada por el cliente, es mejor tener una actitud más ajustada a la realidad de que los requerimientos cambian, la gente se equivoca, se olvida, se da cuenta de lo que quiere cuando lo ve, y no por un contrato. En lugar de depender de un contrato, es mejor tener una comunicación frecuente y colaborativa con el cliente para adaptarse a los cambios y lograr una entrega exitosa del producto.

12 principios del manifiesto ágil

1) Nuestra mayor prioridad es satisfacer al cliente: Esto está ligado al valor 3 y 4, debido a que un equipo ágil desarrolla y entrega funcionalidades en el orden especificado por el Product Owner. Es ese rol quien determina la prioridad e importancia de cada funcionalidad o Feature, de manera que en cada incremento o Release del producto, se esté retornando al cliente el mayor valor posible. Esto está soportado por el hecho de trabajar con User

Stories, las cuales plantean necesidades que tienen un valor significativo para los usuarios del software. El equipo de desarrollo se asegura de estar respondiendo a esas necesidades.

2) Aceptar que los requisitos cambien, incluso en etapas tardías de desarrollo: Ligado a los valores 3 y 4, donde se plantea que el equipo ágil está abierto al cambio, al no seguir un plan estructurado que se confecciona en etapas tempranas, como plantea la metodología tradicional. Esto permite que el equipo sea flexible, y responda siempre ante las necesidades cambiantes de los clientes y/o usuarios.

3) Entregar software funcional frecuentemente: Esto se relaciona al valor 2, donde es imprescindible que un equipo ágil al finalizar cada iteración, haya sido capaz de transformar uno o más requerimientos en código desarrollado completamente, testeado y potencialmente desplegable. Es importante destacar que ese código pueda ser desplegado (que cumpla con la DoD), debido a que se debe asegurar que cumpla con lo que el usuario/cliente necesita, de forma que se le entregue el mayor valor posible tras cada iteración, y no entregar una funcionalidad desarrollada parcialmente.

4) Técnicos y no técnicos trabajando juntos durante todo el proyecto (responsables de negocio, desarrolladores, diseñadores): Valores 1 y 3, y lo que plantea es la gran importancia de la participación del rol conocido como "Product Owner", en conjunto con el equipo de desarrollo de software. Este rol, que generalmente ocupado por una persona del "lado del cliente", debe ser alguien capaz de tomar decisiones sobre el producto, y que además sea capaz de representar el interés y necesidades comunes de todos los usuarios a los que estará destinado el desarrollo del software. Además, generalmente es el encargado de escribir las User Stories, las cuales en cierta forma sirven de recordatorio y guía para el equipo de desarrollo, para conocer qué es lo que deben implementar. El PO se suele convertir en el talón de Aquiles en el proceso de desarrollo del software y siendo uno de los principales motivos por los que se fracasa en los enfoques ágiles.

5) Desarrollamos proyectos en torno a individuos motivados: Ligado al valor 1, se asume que aquellos individuos que participan en un proyecto de desarrollo del sw deben estar lo suficiente motivados y comprometidos con los objetivos de este. Los participantes de un proyecto se deben ver entre sí como a un único equipo con objetivos comunes, donde las personas que asumen diferentes roles intentan colaborar entre sí para cumplir con esos objetivos, y no se limitan a cumplir con sus tareas y delegar los problemas en el otro.

6) El método más eficiente de comunicar información es con conversaciones cara a cara: Ligado con el valor 1, este principio tiene un sustento psicológico, el cual expresa que las palabras expresadas como sonido o texto, sólo representan un bajo porcentaje de la comunicación que se desea lograr hacia otra persona, siendo que la mayor parte está ligada a gestos, expresiones faciales, movimientos corporales, retroalimentación rápida, etc.

7) La mejor métrica de progreso es el software funcionando: Ligado con el valor 2, plantea que la mejor forma de demostrar el avance en un proyecto de software es a través de la entrega de software funcional y de valor para el cliente. En las metodologías tradicionales existen muchas métricas y documentaciones sobre las cuales se basan para medir el avance de un proyecto, pero poco sentido tiene tener una excelente documentación del proyecto y producto, si el producto en sí no refleja el valor que se intenta entregar.

8) Los procesos ágiles promueven el desarrollo sostenible: Ligado con los valores 2 y 4, esto apunta a la estabilidad en equipos de trabajo a lo largo del tiempo, lo cual se traduce en una mejora continua de relaciones y formas de trabajo del equipo. Aunque en la realidad esto se ve afectado en gran forma por la gran migración de trabajadores entre diferentes empresas del mercado laboral informático, la filosofía ágil promueve que estos equipos se mantengan a lo largo del tiempo, para así mejorar su dinámica de trabajo. Permite que el equipo alcance una madurez tal, en la cual es mucho más fácil realizar estimaciones.

- **Framework ágiles:** Apuntan a ciclos de vida iterativos y de duración fija, si no se llega con lo previsto en la iteración se entrega lo que esté listo

9) La atención continua a la excelencia técnica y al buen diseño mejora la agilidad: Esto se relaciona al valor 2, y manifiesta que la calidad del producto no es un aspecto negociable. Es de suma importancia que a medida que el sw se desarrolla, se atiendan las necesidades primordiales del cliente y se responda a ellas, entregando código de calidad.

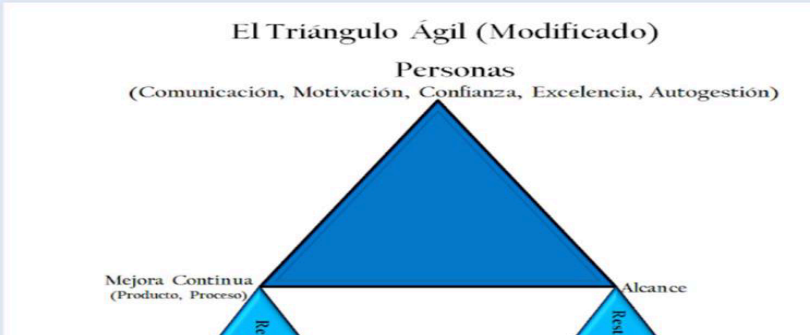
10) La simplicidad es esencial: Ligado a los valores 2 y 3, este principio apunta a que, como equipo de desarrollo, nos centremos en aquellas necesidades que realmente aporten un valor al cliente, y no perder el tiempo en detalles o aspectos que no se traducen en satisfacer requerimientos de los usuarios, ni hacer desarrollos de más que no estén especificados como necesidad del cliente. Es decir, no hacer cosas que el cliente no pidió.

11. Las mejores arquitecturas, diseños y requerimientos surgen de equipos autoorganizados: Ligado al valor 1, este principio expresa que los equipos de desarrollo ágiles a diferencia de organizaciones jerárquicas tradicionales, donde la dirección/gerencia o los jefes son quienes imparten órdenes, y todos deben acatarlas, bajo esta filosofía los integrantes del equipo deben ser proactivos, aportando soluciones, conceptos, herramientas, etc. que sean útiles al equipo. Este principio está sustentado por el empirismo, el cual establece que el conocimiento surge dentro del equipo de trabajo. El equipo toma las decisiones en el momento que ellos creen adecuados.

12) A intervalos regulares, el equipo evalúa su desempeño y ajusta la manera de trabajar: Ligado a los valores 1 y 4, basado en dos pilares empíricos (inspección y adaptación), los cuales permiten a los equipos que cada determinados períodos de tiempo evalúen distintos aspectos concernientes a su desempeño, tales como variaciones de personal en el equipo, tecnologías que no han funcionado como se esperaba, cambio de ideas de usuarios, competencia laboral, riesgos, nuevas tecnologías en uso y reflexionar sobre oportunidades de mejoras en general. Un equipo ágil debe inspeccionar su forma de desempeño, para adaptar aquellos aspectos que sean necesarios corregir, mejorar o incorporar al equipo, de manera que el comportamiento del equipo como un todo mejore, y de esa forma se pueda responder a las necesidades cambiantes de los clientes.

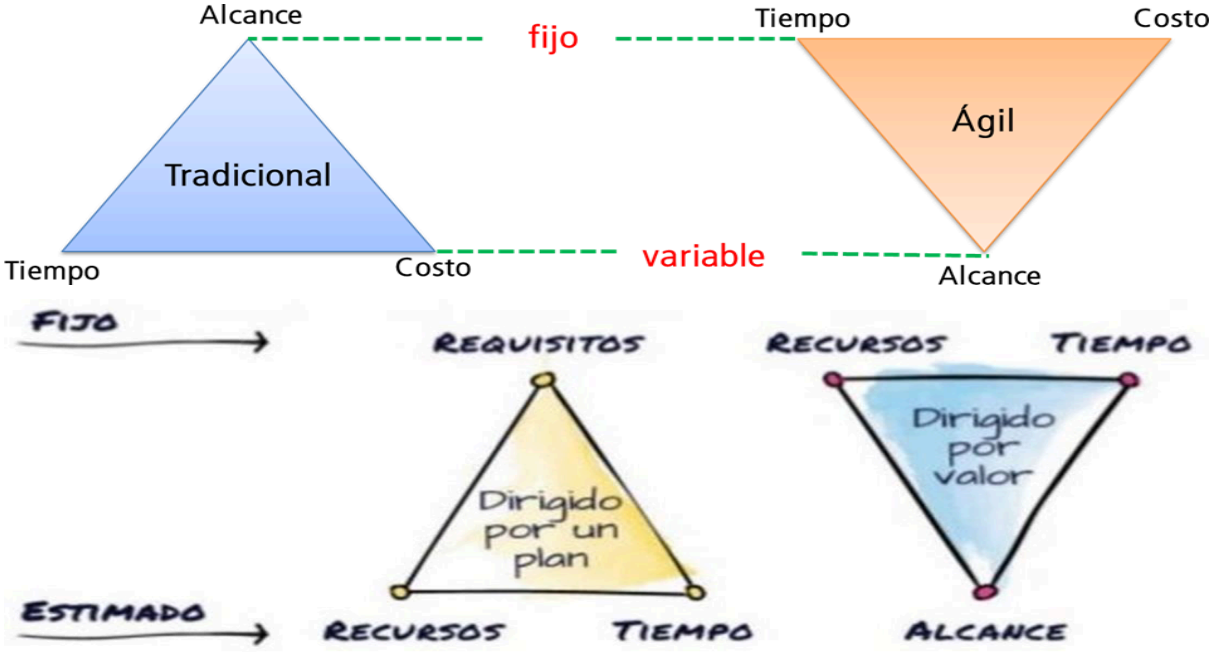
Triángulo ágil y comparación de enfoques

Apunta a lo que realmente tiene valor (sw funcionando) por lo que se centra más en la calidad de producto, es decir, en la calidad intrínseca (las características de calidad que



nosotros le ponemos al producto para que el producto pueda satisfacer al cliente) y la calidad extrínseca (el valor que el producto tiene para el cliente).
Pone a las personas en el lugar más importante ya que las personas son las que hacen el software.

En el enfoque ágil, se trata de aceptar la realidad en la cual el alcance (requerimientos) del producto puede cambiar a lo largo del desarrollo, y no es posible definirlos al comienzo de este. Entonces, el enfoque trata de dejar el tiempo y los recursos fijos, teniendo en cuenta "cuánto software" le puedo entregar en tanto tiempo y con tal equipo de desarrollo.



Por otro lado, en el enfoque tradicional, el alcance se fija y en función de eso derivan los recursos (costo) y el tiempo. El cliente desea controlar el alcance, ya que este comprende lo que él realmente necesita. Luego el cliente pregunta por el tiempo y el costo, los cuales son derivados por el equipo. Por esta razón es tan costosa incorporar cambios de requerimientos, ya que se estaría modificando el alcance y como consecuencia los recursos y el tiempo.

Requerimientos ágiles

El enfoque ágil no solo plantea una gestión diferente del proyecto, sino que a su vez una definición de requerimientos distinta. Dentro de la ingeniería de requerimientos, existen dos procesos: El desarrollo de requerimientos y la gestión de los requerimientos.

- La gestión está vinculada en mantener los requerimientos íntegros, bien identificados y trazados y qué al momento del cambio, que todos aquellos lugares en donde se encuentre se actualicen. Esto ocurre recurrentemente al trabajar con un producto de software.
- El desarrollo de requerimiento es la parte que se centra en identificar, especificar y validar los requerimientos.
    - El 50% de los requerimientos es emergente
    - El otro 50% desconocido se divide en partes iguales entre pasados por alto o conocidos

Los requerimientos ágiles se basan en tres pilares:

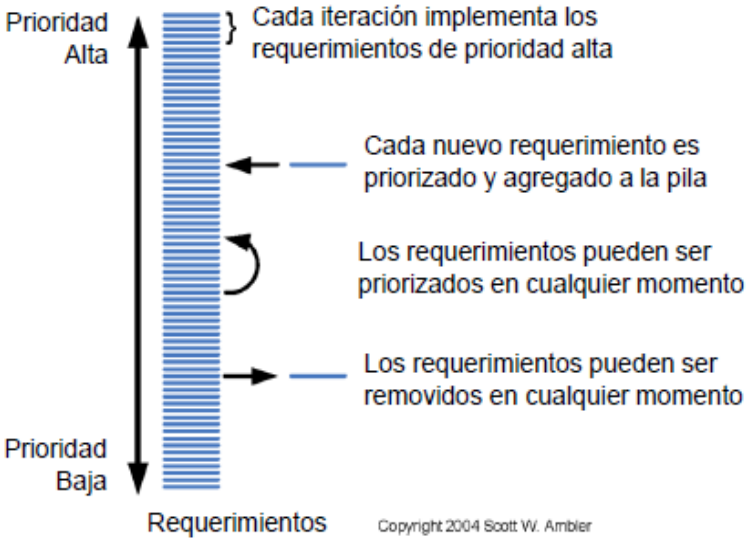
- **Usar el valor para construir el producto correcto:** Lo único importante es dejar contento al cliente con sw funcionando que le sirva, por lo que, el foco de este enfoque de gestión ágil apunta a construir valor de negocio. Es decir, el sw que creamos es para ayudar a alguien más a cumplir con sus objetivos y trabajo, por eso decimos que debemos enfocarnos en construir el producto correcto que le dé valor al negocio.
- **Determinar que es "sólo lo suficiente":** Hace referencia a que los requerimientos deben ser encontrados de a poco, no buscar todos los requerimientos desde un inicio y quedarnos solo con eso. Este pilar postula que se tiene que empezar con lo mínimo necesario para generar realimentación y alimentar el empirismo y a partir de allí, se avanza continuamente. Los requerimientos se expresan en forma de historias de usuario, que son breves descripciones de los requisitos del cliente que se describen en lenguaje natural y se enfocan en las necesidades del usuario. Las cuales son utilizadas para definir el alcance de cada iteración del proyecto y se priorizan en función del valor que aportan al producto. En este pilar hacemos foco en la idea de trabajar par a par con el cliente (1er y 3er valor del manifiesto ágil)
- **Usar historias y modelos para mostrar que construir:** La parte de desarrollo de requerimientos está muy enfocada a la idea de construir junto con el cliente, respetando el principio ágil de "técnicos y no técnicos trabajando juntos". Se busca trabajar con un referente que esté del lado del negocio, que tenga el conocimiento del producto y que trabaje junto con el equipo. Historias de usuario.

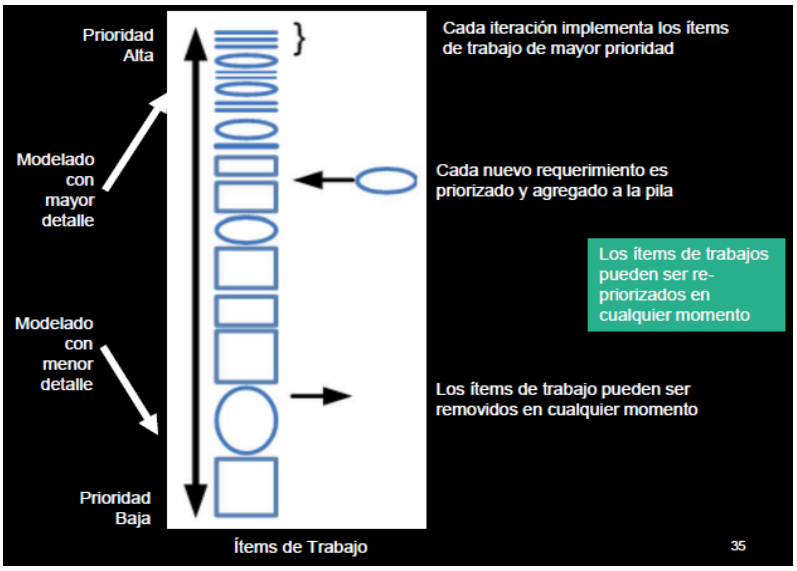
BRUF - Big Requirements Up Front (Definición exhaustiva de requisitos iniciales): En una primera instancia de un proyecto de software, desarrollar sólo unas pocas funcionalidades de gran valor e importancia para el cliente, puede cubrir gran parte de sus necesidades, a medida que se desarrolla el resto de los requerimientos.
Además, es prácticamente imposible conocer al 100% todas las necesidades de los usuarios, ya que ni ellos saben concretamente qué necesitan, ni cómo explicar de forma abarcadora y completa sus necesidades. Por otro lado, esas necesidades que tienen los usuarios al momento de ser relevadas, con el tiempo probablemente irán cambiando, con lo cual, la priorización es un factor fundamental en esa etapa, donde el equipo en conjunto con el Product Owner, deberán ser capaces de decidir cuáles son aquellos requerimientos indispensables para el negocio, de manera que, si se implementan esas necesidades, se puede entregar una nueva versión del producto que sea de valor para el cliente. Los productos "Exitosos" también tienen un desperdicio significante, la frecuencia de uso de determinadas funciones es de:

- 45%: Nunca
- 19%: Rara vez
- 16%: A veces
- 13%: A menudo
- 7%: Siempre

Product backlog (PB): Es una lista priorizada de características y requerimientos del software. Es el Product Owner quien se encarga de definir esta lista y su orden. En ágil es el cliente el encargado de esta priorización, lo que lleva a que se construya el producto deseado.

Con esta situación se permite la compensación de no tener documentos de requerimientos formales escritos, detallando todos los puntos. Ágil ofrece una compensación al afirmar que la mejor comunicación es el cara a cara y un cliente partícipe en el equipo.





Requerimientos just in time: La gestión ágil de requerimientos sostiene los requerimientos justo a tiempo, asociados a los principios de eliminar el desperdicio y diferir compromisos, dentro de la filosofía Lean. Se empieza con una visión del producto y con algunos requerimientos identificados. A partir de esta base, se comienza a construir el producto y se incorpora al cliente para obtener retroalimentación. El Product Backlog nunca está al 100%.

En el enfoque tradicional se aplica el “Up Front Specification”, mediante el cual se especifica todos los requerimientos antes de comenzar el desarrollo. Esto implica que un error en la captura de requerimientos tiene un alto impacto. Pero los requerimientos en ambientes ágiles al ser consensuado continuamente con el cliente, el impacto que se tiene es menor.

Los agilistas no escriben una ERS, pero lo compensan con la disponibilidad del cliente para resolver las dudas. Lo que debemos entregarle al cliente es valor de negocio, no características de sw. El sw es el medio por el cuál nosotros le entregamos valor de negocio.

Tipos de requerimientos:

- **De negocio**: Son los de más alto nivel, son aquellos en términos de la visión del negocio – Ejemplo: Disminuir X% de tiempo invertido en procesos manuales.
- **De usuario:** Tienen que ver concretamente con los requerimientos del usuario final – Ejemplo: Realizar consultas en línea del estado de cuenta de los clientes. → User Stories.
- **Funcionales:** Podríamos relacionarlo 1 a 1 con los CU en la gestión tradicional – Ejemplo: Notificaciones vía mail.
- **No funcionales:** Suelen ser los más ocultos, que el cliente no tiene en claro y se deben especificar. Un RNF puede hacer que el sw que se está desarrollando no sirva (importancia funcional) – Ejemplo: Formato de un PDF.

- **De implementación:** Se suele decir que están dentro de los no funcionales, pero tienen que ver con cuestiones de restricción o implementación específicos – Ejemplo: Servidores en la nube.

La gestión ágil de requerimientos implica trabajar junto a técnicos y no técnicos entendiendo las necesidades y el negocio para construir un sw que de valor. Y luego, junto a los usuarios descubrir cuál es la mejor forma de satisfacer esas necesidades (Product Backlog) al entregar al cliente la característica podemos obtener feedback el cual nos ayudará a refinar el PB. Las US (User Stories) no sirven para especificar requerimientos de sw, sirven para identificar requerimientos de usuario. El foco de la gestión ágil de requerimientos está puesto sobre el usuario y sus necesidades.

- User Stories → Están ubicadas dentro de los requerimientos de usuario con un foco claro hacia el negocio.
- Casos de Uso → Están ubicados en requerimientos de software.

Tenemos en cuenta que vamos a tener una relación con los **principios ágiles** al utilizar la gestión ágil, por ende, estos son:

1. Los cambios van a existir todo el tiempo de forma constante: Todo el tiempo habrá cambio, por lo que se busca gestionar los requerimientos orientados a poder aceptar el cambio. Si el PO, el cliente o los stakeholders relacionados con el producto no plantean cambios es porque el producto no está siendo utilizado.
2. Debemos tener una mirada de todos los que están involucrados: Todos aquellos que tengan algo para decir del producto (Stakeholders: No son todos los que están.) Siendo el PO el representante de todos los involucrados.
3. El usuario dice lo que quiere cuando recibe lo que pidió: En base a esto las iteraciones son cortas y las entregas continuas son esenciales, permitiendo obtener retroalimentación y un enriquecimiento para futuras entregas y para una retroalimentación significativa.
4. No hay técnicas ni herramientas que sirvan para todos los casos: Se deberá identificar la herramienta adecuada para cada situación.
5. Lo importante no es entregar una salida, un requerimiento, si no, un resultado, una SOLUCIÓN DE VALOR: Con esto se hace referencia a que sea útil para el negocio o que el cliente minimice el desperdicio.

Principios ágiles relacionados a los requerimientos ágiles: Los más importantes son el principio 4 y 6. Ya que si recordamos lo expresado anteriormente podemos darnos cuenta qué son los que más se tienen en cuenta para esta etapa



User Stories

Son una técnica para trabajar requerimientos de usuario en ambientes ágiles. La US contiene una descripción corta de una funcionalidad que se espera del producto valuada por un usuario del sistema.
Funciona como un recordatorio para el equipo de desarrollo y el Product Owner de que tienen que conversar acerca de la necesidad en términos de negocio para la construcción de una característica del producto que aporte valor. En otras palabras, es un mecanismo para diferir una conversación. Es un requerimiento a nivel de usuario, no al nivel de sistema, por lo que se encuentra a un nivel de abstracción más elevado. Es un ítem de planificación que no acompaña al producto durante todo el ciclo de vida. Para eso está el manual de usuario. Las US son escritas por el Product Owner. Internamente, las User Stories se consideran verticales dentro del producto, ya que pueden incluir aspectos desde diseños de interfaces hasta diseños de tablas en la base de datos.

La problemática que motivó en la definición de esta técnica es la dificultad de comunicar lo que hará el sistema a todas las personas afectadas. Cada una tiene un interés respecto a esta información. La definición de los requerimientos es lo que más afecta la calidad del sistema final y es lo más complejo de definir con detalle debido a la incertidumbre y cambio constante que sufren, que se da por el problema de comunicación. La dificultad se basa en qué se escribe y cuándo. Las US se empiezan y se terminan en la iteración.
Podemos decir que las user stories son multipropósito debido a que son:

- Una necesidad del usuario.
- Una descripción del producto.
- Un ítem de planificación para saber lo que entra en una iteración.
- Token para una conversación.
- Mecanismo para diferir una conversación.

Partes de una User Stories - Triple C:

- **Conversation - Conversación:** Es la parte más IMPORTANTE que no queda escrita en ningún lugar. En esta conversación, es donde se obtienen todos los detalles para que el equipo pueda trabajar esa historia – Es la parte no visible. Es importante ya que nos permitirá dar límites o establecer lineamientos en base a está conversación con el PO o quien conozca el equipo de desarrollo.
- **Card – Tarjeta:** Es donde se escribe/expresa algo que os indique cual es la US, de que se trata y su valor de negocio – Parte visible de la US.
- **Confirmation – Confirmación:** Son aquellas condiciones que se tienen que dar para dar por satisfecha la US. Se encuentra dentro de la tarjeta – Son las pruebas de usuario.

En la tarjeta, se define:

- **Nombre del rol:** Representa quien está realizando la acción o quien recibe el valor de la actividad.
- **Actividad:** Representa la acción que realizará el sistema.

- **Valor de negocio:** Es la más importante y representa el por qué es necesaria la actividad. Esto justifica la razón de que el Product Owner escriba las US. Permite priorizar el desarrollo.
- **Frase verbal:** No es obligatoria, es una forma corta de referenciar la US. Representa la funcionalidad que se expresa en la User Story.
- **Criterios de aceptación:** Son diferentes criterios sobre la actividad que se necesita implementar, que permiten definir los límites de la U.S. para que el equipo tenga una mejor visión de esta. Además, facilitan las tareas de desarrolladores y testers para probar la funcionalidad.
- Se deben redactar **independientes** a la implementación (alto nivel de abstracción).
- **Pruebas de aceptación:** Acompañan a la tarjeta en la parte trasera, y es donde se capturan todos los detalles (que se manifiestan en la conversación) de la User, permitiendo determinar cuándo una historia está completa. Esto servirá a los desarrolladores para probar si la implementación ha sido realizada de forma correcta y completa. Las pruebas pueden agregarse o quitarse en cualquier momento.

Forma de expresar una US: Basados en la tarjeta, tenemos la siguiente nomenclatura definida: Como <Nombre del rol> yo quiero/puedo <actividad> de forma tal que <valor de negocio que recibo>.
Esta forma de expresar una US busca contestar las siguientes preguntas:

- **WHO?:** Quién necesita esto.
- **WHAT?:** Que se necesita.
- **WHY?:** Por qué o para qué → Haciendo referencia al valor de negocio

Story Points: Son unidades de medida relativas (no importa el valor en sí, sino su comparación), las cuales miden el tamaño de un producto, NO de una medida basada en el tiempo (por ejemplo, horas hombre). Esto se debe a que estimar basándonos en el tiempo, conduce a muchos errores. Lo relativo surge a partir de que se comprobó que las personas no saben estimar en términos absolutos, sino que les resulta útil la comparación de tamaños.
Hay 3 aspectos que **determinan** el tamaño de una U.S.:

- **Complejidad:** Hace referencia a una dificultad inherente a la funcionalidad, que no permiten dividirla en U.S. más pequeñas. Por ejemplo, algoritmos complejos, cálculos, gran cantidad de campos, muchas validaciones, demasiados filtros, etc.
- **Esfuerzo:** Es el tiempo en horas de trabajo que requerirá la implementación.
- **Incertidumbre;**
    - **Técnica:** Cuando se tiene dudas en el dominio de la solución, por ejemplo, el uso de una cierta tecnología no muy conocida por el equipo.
    - **De negocio:** Cuando hay incertidumbre respecto a cómo interactuará el usuario con el sistema, lo cual es causado por falta de conocimiento del dominio del negocio.

Modelado de roles: Se trata de modelar los roles dentro del sistema para lograr identificar aquellos que probablemente formen parte de las US, y en base a ellos, encontrar las US.

Criterios de aceptación: Son la base para que luego se construyan los casos de prueba. Ya que son aquellas cuestiones que definen los límites de la US. Ayudan a que el PO

responda los criterios que necesita para que la US provea valor de negocio y también a que el equipo tenga una visión compartida de la US.

- Definen límites para una US.
- Ayudan a que los PO respondan lo que necesitan para que la US provea valor (Req. Funcionales mínimos).
- Ayudan a que el equipo tenga una visión compartida de la US.
- Ayudan a los desarrolladores a saber cuando parar de agregar funcionalidades a una US. No van detalles dentro de los CA.

Son aquellos que definen una intención y no una solución, que son independientes de la implementación y que son relativamente de alto nivel. Su redacción deberá ser clara y tienen que haber validaciones concretas.

- Puede → Opcional.
- Debe → Obligatorio.

Son las consideraciones que define nuestro usuario a la hora de hablarnos de cómo imagina él esa funcionalidad, no hay limitación en la cantidad de criterios.
Se escriben a nivel de usuario, debiendo ser objetivos y verificables

Pruebas de aceptación: Son declaraciones de intención de que hay que probar. El conocimiento del negocio está inmerso en las pruebas de aceptación. Se prueban escenarios exitosos y escenarios que fallen. En las pruebas de aceptación se encuentran las más importantes, ya que no es posible escribir todas las posibles pruebas de aceptación. El Product Owner acepta la User como implementada si todas las pruebas de aceptación se cumplen. Expresan detalles resultantes de las conversaciones entre clientes y desarrolladores; suelen usarse para completar detalles de la US. Es importante mencionar que las mismas deben escribirse antes que la programación empiece, y que se recomienda que las escriba el cliente

DoR (Definition of Ready) – Definición de Listo: Es una medida de calidad que determina si la User Story está en condiciones de entrar a una iteración de desarrollo. Para que la US pueda ser implementada, mínimamente debe satisfacer el INVEST Model. Entre el equipo de desarrollo y el PO pueden definir características o condiciones extras al INVEST para que una US se considere lista para entrar a la iteración de desarrollo.

DoD (Definition of Done) – Definición de Hecho: Determina si la US está terminada. Son los criterios que establecen cuándo una User Story ha sido implementada de forma correcta y completa, de forma que puede ser mostrada al Product Owner, para que éste decida si se pasa a producción o no.
En el momento que una U.S. cumple con todo lo pactado en esta Definición de Hecho, se considera que esa US está lista para ser desplegada y aportar valor al cliente. Esto permite crear transparencia, debido a que proporciona a todo el equipo una comprensión exhaustiva de qué trabajo se realizó, para lograr ese incremento.
En la práctica consta de un Checklist de condiciones que deben cumplirse. Puede ocurrir que una organización tenga un estándar de DoD, entonces los miembros del equipo deberán adaptarse a ese estándar. O bien, que la organización no posea un estándar, con lo cual el equipo deberá confeccionar y crear una DoD adecuada para el producto.

Modelo INVEST

Es un modelo de calidad que nos ayuda a verificar si la User cuenta con las características de la calidad mínima para satisfacer la definición de Ready y poder ingresar en una iteración de desarrollo.

- **Independent – Independiente:** Calendarizables e implementables en cualquier orden. Otorgamos libertad al PO de que este priorice la US para que se puedan desarrollar en cualquier orden.
- **Negotiable – Negociable:** El “que” no el “como”. La US debe estar escrita en términos de qué necesita el usuario, no de cómo se implementará.
- **Valuable – Valuable:** Debe tener valor para el cliente. Debe estar el para qué, es decir, debe tener valor de negocio.
- **Estimatable – Estimable:** Para ayudar al cliente a armar un ranking basado en costos. Se asigna un peso a la US indicando cuánto tiempo llevará terminarla.
- **Small:** Debe ser “consumidas” en una iteración. Se comienza y termina una característica en una iteración, no se deja trabajo a medias. Depende mucho del equipo, la experiencia de este, la capacidad de trabajo y la duración de la iteración.
- **Testeable:** Demostrar que fueron implementadas. Relacionado con las PA, permitiendo demostrar que la US efectivamente fue implementada cumpliendo con los CA definidos.

Diferentes Niveles de abstracción

- **Epic – Épica:** US muy grandes que no están en un lugar del Product Backlog o que todavía no deben ser implementadas No cumple con el modelo INVEST, o cuando no se puede ejecutar en una iteración, debe ser dividida.
- **Theme - Tema:** Agrupa US relacionadas a un mismo tema. Su nivel de abstracción es más grande que el de una épica.
- **Spike:** Son un tipo especial de US, son utilizadas para quitar el riesgo e incertidumbre de una US u otra faceta del proyecto que nosotros queremos investigar. Inversión básica para familiarizar al equipo con una nueva tecnología/dominio.
    - Analizar un comportamiento de una historia compleja y poder así dividirla en piezas manejables.
    - Ganar confianza frente a riesgos tecnológicos, investigando o prototipando para ganar confianza.
    - Frente a riesgos funcionales, donde no está claro como el sistema debe resolver la interacción con el usuario para alcanzar el beneficio esperado.

Las Spikes a su vez pueden dividirse entre: Técnicas y Funcionales

- **Técnicas:** Utilizadas para investigar enfoques técnicos en el dominio de la solución. Evaluar performance potencial. Decisión hacer o comprar. Evaluar la implementación de cierta tecnología. Cualquier situación en la que el equipo necesite una comprensión más fiable antes que comprometerse a una nueva funcionalidad en un tiempo fijo.

- **Funcionales:** Utilizadas cuando hay cierta incertidumbre respecto cómo el usuario interactuará con el sistema. Usualmente son mejor evaluadas con prototipos para obtener retroalimentación de los usuarios o involucrados.
- Hay US que requieren ambos tipos de spikes

Lineamientos para Spikes:

- Estimables, demostrables y aceptables → Deben cumplir el Criterio de Ready.
- La excepción, no la regla:
    - Toda historia tiene incertidumbre y riesgos.
    - El objetivo del equipo es aprender a aceptar y resolver cierta incertidumbre en cada iteración.
    - Los spikes deben dejarse para incógnitas más críticas y grandes.
    - Utilizar spikes como última opción.
- Implementar la spike en una iteración separada de las historias resultantes:
    - Salvo que el spike sea pequeño y sencillo y probable de encontrar una solución rápida en cuyo caso, spike e US pueden incluirse en la misma iteración. De lo contrario se realizan en un sprint anterior al que se va a implementar la US en la que se tiene incertidumbre.

Estimaciones Ágiles y de SW (PPT 4 - 7)

Estimar es el proceso de obtener una aproximación sobre una medida con el objetivo de predecir la completitud y gestionar los riesgos en el contexto de un proyecto. Se relaciona con los objetivos del negocio, compromisos y control.
Según McConell una estimación en el contexto del software es una predicción de cuánto tiempo durará o costará un proyecto. Es considerada una de las actividades más complejas en el desarrollo de software. El cliente siempre pregunta cuanto y cuando, el tiempo y el dinero son las variables que más se intentan estimar
Las estimaciones no son planes ni compromisos, sino que sirven de base para planificar, pero el plan final no tiene por qué coincidir con lo estimado.
Una estimación debería venir acompañada con la probabilidad asociada de que eso se cumpla. Puede haber ciertos errores en las estimaciones:

- Información imprecisa acerca del software a estimar o acerca de la capacidad para realizar el proyecto.
- Demasiado caos en el proyecto (mal definido el proyecto).
- Imprecisión generada por el proceso de estimación.
- Una de las fuentes más común es omitir actividades necesarias para la estimación del proyecto tales como, requerimientos faltantes, licencias, reuniones, revisiones.
- Se olvidan del detalle que planear no es estimar y estimar no es planear, sino que las estimaciones son la base de los planes, pero los planes no tienen que ser lo mismo que lo estimado.
- A mayor diferencia entre lo planeado y lo estimado, mayor el riesgo.

Siempre se desea saber muy al inicio del proyecto cuál será el costo, el tiempo que llevará, entre otros aspectos y como aún no se conoce con precisión cuál será el alcance del producto resulta complejo predecir con exactitud estos aspectos, es por ello que a medida que avanza el proyecto, las estimaciones son más certeras, dado que se cuenta con

mayor información sobre la cual realizar las estimaciones. (Es por ello que se suele decir que típicamente la primera estimación difiere hasta un 400%).
Al inicio del desarrollo hay una estimación errónea de 2 a 4 veces más, esto se debe a:

- Al inicio del proyecto tenemos poca información y demasiada incertidumbre, lo que aumenta el riesgo de generar estimaciones erróneas.
- Las estimaciones son de naturaleza optimista, por eso las estimaciones deben ser recalculadas conforme se avanza con el proyecto.
- Se suele decir: “el que no sabe estimar, estima muchas veces, y el que no sabe planificar, planifica muchas veces”.
- El compromiso es después de la planificación, la planificación toma como entrada la estimación, y la salida de la planificación es que logra establecer el compromiso con el cliente
- Se suele hacer un 20% de más / menos, ejemplo 12 a 16 semanas.
- Igual siempre tiene que haber una transparencia de porqué pasó lo que pasó, sin "peinarse para la foto"

Frente a la posibilidad de estimar mal y generar enojo por parte del cliente tenemos dos reacciones posibles:

- Subestimar: Se simplifica sin considerar todo el trabajo que hay que hacer, por miedo a que si el valor es más alto el cliente lo rechace.
- Sobreestimar: Se infla de más la complejidad del producto, tendiendo a confundir con compromisos o planificación.

Estimar NO es planear y no necesariamente nuestro plan tiene que ser lo mismo que lo que estimado. Si bien está sirve para planificar, no tienen que seguirlas ya que no son compromisos.

Técnicas de estimación

Son formas/técnicas que nos ayudarán a disminuir/acotar la incertidumbre. No van a eliminar la incertidumbre, pero nos situará en contexto. Cada técnica tiene sus ventajas y desventajas, por los cual, es importante seleccionar la adecuada para cada proyecto en función de sus requisitos y características.

- **Métodos basados en la experiencia:** Se basa en la experiencia pasada del equipo en proyectos similares. Para determinar la estimación de: Esfuerzo y tiempo requerido p/ un nuevo proyecto.
    - **Datos históricos:** Tomo como base la experiencia de otras personas y proyectos. Lo cuál no es acorde a la gestión ágil ya que explicita que la experiencia no se puede extrapolar a otros equipos.
    - **Juicio Experto:** Técnica basada en la experiencia y conocimiento de expertos en la materia para determinar la estimación. Sujeta a la subjetividad y experiencia de los expertos. Las estimaciones pueden variar dependiendo de las características del experto involucrado. Estimar solo tareas con granularidad aceptable (no alta).
        - **Puro:** Un experto estudia las especificaciones y hace su estimación. Basado fundamentalmente en sus conocimientos.
        - **Delphi:** Se realiza de forma grupal donde indicarán lo que costará el desarrollo (esfuerzo y duración). Está estimación suele ser mejor que

la individual. Es tomado como base para el Poker Planning. Múltiples rondas hasta que la estimación converja de forma razonable.

- **Analogía:** Basada en datos históricos, pero se realiza una abstracción de aquellos parecidos o que concuerdan con el proyecto con el que se está trabajando.
- **Otros:**
    - **Métodos basados exclusivamente en recursos.**
    - **Métodos basados exclusivamente en el mercado.**
    - **Métodos basados en los componentes del producto o en el proceso de desarrollo.**
    - **Métodos algorítmicos.**

Problemas y errores en estimaciones

- Estimar demasiado pronto.
- Resistencia a las reestimaciones.
- Estimaciones que buscan precisión.
- Estimaciones hechas por gente distinta a la que hace el trabajo.
- Estimaciones que pretenden ser absolutas (inflexibles).
- Si el proyecto no está organizado, el nivel de conocimiento sobre el producto es menor, provocando que las estimaciones sean imprecisas.
- Si no se tiene en claro los recursos con los que la empresa cuenta para desarrollar el producto ni las capacidades de estos, las estimaciones no suelen ser buenas.
- Las técnicas de estimaciones también pueden ser fuente de error, para solucionar esto se suele emplear más de una técnica y comparar sus resultados. Ante resultados muy distintos, es índice de que hay un error.

Estimación en ambientes ágiles

Son presunciones numéricas asociadas a una probabilidad de que cierto evento ocurra. La filosofía ágil plantea que quien estima, es quien debe hacer el trabajo, con lo cual es el equipo de desarrollo quien realiza las estimaciones, de forma colectiva (menos el Product Owner, no participa en estimaciones). Al ser de esta manera, la estimación se ajusta mejor, debido a que se generan debates entre los miembros del equipo, lo cual puede abrir puntos de vista no detectados por otros, haciendo que ese proceso de estimar se vuelva lo más preciso posible (a diferencia de las metodologías tradicionales, donde se estima individualmente y se depende sólo de ese miembro).

En el enfoque ágil, el esfuerzo destinado para estimar es bajo, pero amparándose en el concepto de que, al destinar poco esfuerzo en la estimación, se logra acercarse bastante a la realidad, la cual no sigue una relación lineal: si destinamos demasiado esfuerzo a este proceso, no obtendremos una precisión casi perfecta.
Es por esto, que, en este enfoque, el tiempo de estimación se define y asigna con anterioridad para no utilizar más esfuerzo del necesario. El gran beneficio del proceso de estimación es justamente “hacerlas”. Al estimar, se enriquece el trabajo del equipo, lo cual mejorará futuros procesos de estimaciones. Además, este proceso ayuda a determinar la factibilidad del trabajo planificado en etapas tempranas.

Lo que se estima en ambientes ágiles, es solo el tamaño de las US (o Features) usando como unidad de medida los Story Points, y teniendo en cuenta 3 aspectos de estas: incertidumbre, complejidad y esfuerzo. Como la complejidad de las US tiende a incrementarse exponencialmente, se utilizan comúnmente series numéricas de esta índole, siendo una de las más comunes, la serie de Fibonacci.
Al ser estimaciones de tamaños relativos, lo importante en este proceso no es el valor que se asigna a cada US en particular, sino más bien que las estimaciones sean consistentes en el conjunto. Es importante tener en claro que no se estima tiempo, ya que la duración de los Sprints es fija (concepto de Timebox). Por esto, es que no se recomienda estimar el tamaño de las US en horas, para no confundir con el esfuerzo.

Premisas de la estimación relativa: Usualmente las personas no saben estimar en términos absolutos, para ello utilizamos la comparación, ya que es generalmente más rápida para asociar. En este caso en la estimación ágil hacemos uso de las US Canónicas, obteniendo una mejor dinámica y acuerdo grupal, centrando el pensamiento y a su vez permitiendo emplear de mejor manera el tiempo de análisis de las US.
Al estimar una US se utiliza el: Tamaño, Tiempo y Esfuerzo necesario de esta.

Tamaño vs. Esfuerzo: Como el esfuerzo (horas de trabajo) es el factor más significativo del costo de un proyecto, tenemos una gran tendencia a confundir tamaño con esfuerzo y esfuerzo con calendario. La complejidad es independiente de quien haga el proyecto, y el esfuerzo depende específicamente de la persona que lo va a llevar a cabo, de su capacitación, conlleva horas lineales (horas únicamente de trabajo, sin tener en cuenta almuerzo, baño, leer emails, etc. Para una jornada de trabajo de 8 horas, usualmente solo 5.5 o 6 son lineales). Por otro lado, no se debe confundir esfuerzo con calendario, ya que yo puedo decir que puedo tener un trabajo para el viernes, pero en el medio pueden suceder millones de cosas que evitan que cumpla mi objetivo. Un trabajo puede requerir un esfuerzo de x horas, pero no se puede asegurar que para tal día va a estar terminado.

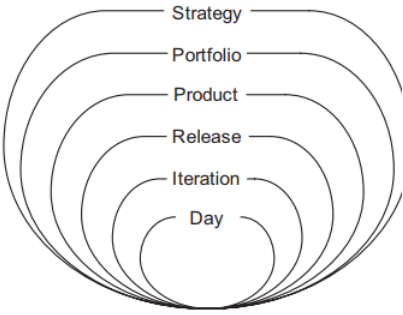
- **Tamaño:** Medida de la cantidad de trabajo necesario para producirla/completarla, y a su vez índica que tan compleja y que tan grande es.

Velocidad: Es una métrica del progreso de trabajo de un equipo. Se calcula (no se estima) sumando el número de Story Points asignados al estimar una US, que el equipo completa durante cada iteración, lo cual implica que la US se ha desarrollado y testeado por completo (DoD), y además el Product Owner la ha aceptado.
Si la US se completó parcialmente, no se cuentan sus Story Points en esa iteración.
Esto genera que la velocidad del equipo baje en esa iteración, pero en la siguiente, aumentará, debido a que en unos pocos días probablemente se complete la US pendiente, y se pueda continuar con otras US planificadas en la iteración.
Esto implica que el equipo no se enfoque en la velocidad de cada iteración, sino más bien, en el promedio de velocidades a lo largo de sucesivas iteraciones. Esto brindará una mayor certeza acerca de la medida de progreso de trabajo del equipo.
Esta métrica permite retroalimentar el proceso de estimación, ya que un equipo maduro y con mucho trayecto, puede permitirse estimar cuántos puntos de historia puede desarrollar en la siguiente iteración a través del cálculo de velocidad. No se debería reestimar porque no se detecta un progreso como el esperado (velocidad del equipo), sino que se debe dejar que la velocidad a lo largo de sucesivas iteraciones equilibre esas inexactitudes.

Tamaño: Dado que el tamaño es lo que se compara, lo definimos como la medida de cuán compleja y grande es una US, además de cuánto trabajo es requerido para hacer o completar una Feature/US. También el tamaño es función de la incertidumbre de dicha US, definida la incertidumbre como la falta de información.
El tamaño no es lo mismo que el esfuerzo (el "quien" define esta diferencia), ya que este último indica las horas humano lineal requeridas para el desarrollo de la US. Por lo tanto, una US de mismo tamaño puede implicar esfuerzos diferentes en función de la persona que se esté evaluando, debido a que el esfuerzo varía dependiendo de las habilidades, conocimientos, experiencia, familiaridad con el dominio, etc.
Para ello contamos con la técnica CONTAR. Contar: Funcionalidades, Requerimientos, Cant. De US, Cant de CU.
Aquí entra en juego la información que tenemos para realizar la predicción/estimación y el nivel de incertidumbre/riesgo que vamos a tener.

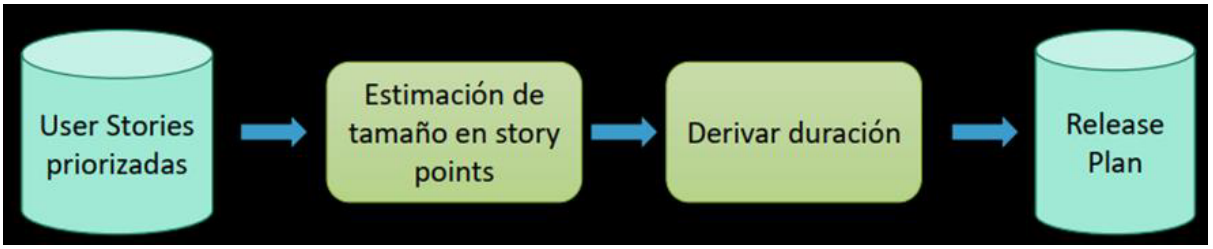
- **Esfuerzo:** Es la cantidad de horas lineales que se necesitarán para construir el sw, no se incluye a las personas, ni calendario, ni las actividades detalladas, solo CUENTO, en estimación, cuantas horas lineales de desarrollo necesitaré.
- **Calendario:** Tras estimar el esfuerzo, se calendariza el esfuerzo y se propone una fecha de entrega.
- **Costos y presupuestos:** Tras estimar, tamaño, esfuerzo y tiempo calendario se comenzara a trabajar con esto.

Planificación



En ambientes ágiles, es necesario que quien planifica el proyecto de desarrollo de software, sea capaz de analizar el contexto del proyecto cada cierto período y hacer aquellos ajustes que sean necesarios, para encauzar los esfuerzos de manera correcta, y poder lograr los objetivos planteados.
"La planificación lo es todo. Los planes no son nada." → esto apunta a que lo importante de planificar, es el proceso en sí. No el hecho de realizar una documentación exhaustiva que respete determinados templates. Sí debe quedar registrado de alguna manera, idealmente bajo templates, pero sólo para servir de ayuda de memoria. Lo importante es el proceso.
Los equipos ágiles afrontan esta situación, planificando al menos en 3 niveles:

1) Planificación de release: Este proceso se da al inicio del proyecto, donde se analiza y determina qué US, épicas o Temas serán desarrollados en una nueva entrega (Release) del producto. Este plan se va actualizando mientras el proyecto transcurre (usualmente al comienzo de cada iteración), de manera que siempre refleja las expectativas actuales sobre qué se incluye en la entrega.
Una vez que se priorizan las US a implementar, se estima su tamaño con Story points, y teniendo en cuenta la velocidad de trabajo del equipo, se deriva la duración de la Release o del proyecto (la cantidad de sprints necesarios para obtener el release).



2) Planificación de Iteración: Al comienzo de cada iteración, donde el PO identifica aquellas US de mayor prioridad que deberán ser implementadas en la iteración, basándose en la iteración anterior ya finalizada. Esto se hace teniendo en cuenta el tiempo (que es una variable constante por ser metodología ágil) y la velocidad de trabajo del equipo (el cual se mantiene constante también). Durante esta planificación, se discute sobre las tareas que serán necesarias para transformar una US en código funcional y testeado.

3) Planificación del día: Este proceso permite coordinar y sincronizar el trabajo del equipo, en reuniones diarias. Haciendo foco en las actividades que deberán ser desarrolladas para lograr cumplir con una tarea específica.

Poker estimation

Es una técnica de estimación en ambientes ágiles publicada por Mike Cohn. Resulta de la conjunción de diferentes métodos como el de juicio experto, analogía y desagregación, basado en que 4 ojos ven más que 2. En contraposición al método de juicio experto tradicional, asume que las personas que desarrollan el producto deben ser aquellas que estimen. El equipo estima su propio trabajo y los miembros opinan sobre lo mismo, donde se mezcla la experiencia y el conocimiento de cada uno con la posibilidad de compartir con el resto del grupo. Un detalle es que el Product Owner participa en la planificación, pero no realiza estimaciones, suele ser el moderador, pero no es necesario que sea así siempre.

Escala: Dado que la complejidad en el software incrementa de forma exponencial, se decide adoptar la serie de Fibonacci como escala para la determinación de los puntos de historia.

- **Valor 1 o 1/2:** Son los valores más bajos y se aplican a una funcionalidad pequeña.
- **Valor 2- 3:** Funcionalidad pequeña a mediana.
- **Valor 5:** funcionalidad media.
- **Valor 8:** funcionalidad grande, donde nos preguntamos si es posible dividir.
- **Valor 13 o más:** Necesariamente hay que dividir ya que no ingresa en el sprint.

Procedimiento:

1. Se define una lista de actividades, módulos, casos de uso, Users stories, etc.
2. Se acuerda cuál de las anteriores es la que tiene menor grado de incertidumbre y se la define como canónica asignándole 1 o 2 story point (es la más simple). Esta User es utilizada como referencia para comparar. Los equipos avanzados podrían elegir canónicas de 3 SP, con el objetivo de tener un margen para hacia arriba y hacia abajo. Por esta razón, también él ½ se incluye en la serie.

3. Cada miembro elige una carta con el valor de la estimación del tamaño de la U.S. (comparado con la U.S. canónica y teniendo en cuenta los 3 aspectos nombrados) y se espera a que todos decidan.
4. Utilizando juicio experto, cada integrante del equipo de desarrollo expone su valor de estimación de forma simultánea, por medio de las cartas, dándolas vuelta a las que estaban boca abajo. En caso de que la estimación difiera, el mayor y el menor estimador justifican sus estimaciones para conocer sus opiniones. Luego el resto del medio aportan sus opiniones. Siempre la escucha es activa, para poder aprender.
5. En caso de que la primera vuelta existiese diferencias considerables, se realiza nuevamente la ejecución del paso 4.
6. Si las estimaciones no convergen, entonces se llega a una estimación intermedia o promedio.

Consideraciones:

- Para realizar una estimación se debe tener en cuenta el criterio de Done y no solo la tarea de programación, es decir todo lo que hay que hacer para terminar una US y poder lidiar con el problema de las actividades omitidas.
- El tiempo para realizar las estimaciones, como cualquier otra actividad en los ambientes ágiles, es time boxing.
- Las US por estimar son las menos posibles, en orden con el principio de diferir compromisos.

Diferencias con las estimaciones en ambientes tradicionales:

Respecto a las tres dimensiones:

- En Agile se tiene fijo el tiempo (lo que dura la iteración) y los recursos afectados para trabajar, dejando variable el alcance. Por lo tanto, al estimar en estos ambientes se responde a cuánto se puede comprometer el equipo, que cantidad de las características de producto se puede construir en la iteración. Se entiende, además, que en estos ambientes el producto no está completamente definido, sino que se tiene una visión de este.
- Tradicionalmente, el alcance es lo que está fijo y el producto se encuentra definido. A partir de esto, se deriva el resto de las variables. Por esta razón en los procesos definidos se estima primero el tamaño, ya que es la base para estimar el resto.

Respecto a la precisión: Las estimaciones en los ambientes tradicionales buscan ser más precisas, debido a que son las bases para luego planificar. Dado que la precisión es realmente cara, la precisión en los ambientes ágiles y en correspondencia con la filosofía Lean, es considerada un desperdicio.
Principal diferencia: Las estimaciones ágiles son relativas, a diferencia de las estimaciones en ambientes tradicionales que son absolutas. En Agile, las características del producto son estimadas utilizando una medida de tamaño relativa dado que las personas no somos buenas realizando estimaciones absolutas, sino que es más fácil y rápido comparar medidas. Por otra parte, la práctica de estimar mediante comparaciones permite obtener una mejor dinámica grupal y pensamiento de equipo por sobre el individuo.
Quien estima: En las metodologías de estimación tradicional, las realiza el líder del proyecto o en ocasiones las realiza un experto, En los ambientes ágiles, es el equipo el que

realiza la estimación. Lo óptimo es que cada uno estime su propio trabajo y de esta manera, es posible disminuir la probabilidad de desviaciones.

Momentos para estimar: En el enfoque tradicional se estima al comienzo del proyecto, luego de la especificación de requerimientos y luego del diseño (debido que ahí es cuando puedo saber cuál es el tamaño y de ahí derivo todas las demás estimaciones). Realmente se estima cada vez que se modifica el alcance, ya que es la variable que se mantiene fija y de la cual se deriva el resto. Las estimaciones a lo largo del proyecto se van haciendo cada vez más certeras. Al comienzo, pueden diferir hasta un 400%, esto se debe a:

- Al comienzo tenemos poca información y alta incertidumbre, por lo que aumenta el riesgo de estimar erróneamente.
- Las estimaciones son demasiado optimistas.
- Decimos que aquel que no saber estimar implica estimar muchas veces.

En ágiles se estima al comienzo de cada sprint.

Gestión de Producto (PPT 5)

Un producto de Software es un artefacto que se construye para satisfacer una necesidad específica en base a ciertos requerimientos. Cuando creamos un producto debemos enfocarnos en aquello que es de valor para el cliente, sabiendo que el 7% de las características del software son siempre usadas. Esto implica, que la mayor parte del valor agregado está concentrado en ese 7%. Alrededor del 80% de las características del software que se desarrolla no se utilizan. El desafío está en encontrar ese 7%.

Las organizaciones que usan el software se enfocan en la parte superficial del software, aquello que consideran significativo y placentero. Usualmente no se fijan si el producto es funcional o confiable. Esa debe ser la base, pero el producto debe proveer una experiencia que cumpla con todas las expectativas del usuario. Nosotros creamos productos para:

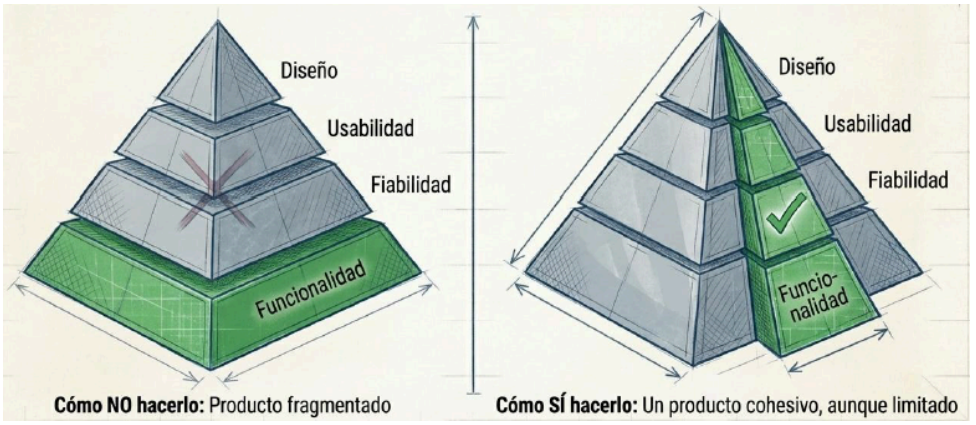
- Satisfacer a los clientes.
- Tener muchos usuarios logueados. Una métrica que se utiliza para saber que tan aceptado es un producto, es por la cantidad de usuarios que lo instalan.
- Para obtener mucho dinero.
- Realizar una gran visión, cambiar el mundo o nuestra diaria.

Evolución de los productos de software

Si tenemos en cuenta lo mencionado anteriormente, podríamos armar una especie de pirámide respecto a la evolución de los productos de sw, pensando en las características del sw. El desafío surge ya que debemos ver cómo hacer para construir un producto de sw donde desechamos el desperdicio y logramos abarcar aspectos que perseguimos al momento de crear el producto (Sea satisfacer a un cliente o la venta del producto, etc). Por lo que al centrarse en el desarrollo del mínimo producto/mínima característica para saber si el producto que construiremos cubrirá las expectativas del usuario o las propias al desarrollarlo → Surge el planteo de la hipótesis: Mínimo desarrollo y mínimo esfuerzo para

luego validarla haciendo que su desperdicio tienda a 0, evitando malgastar esfuerzo/energía en algo que no agregara valor/beneficio al producto.
La productividad de un Startup no puede medirse en términos de cuánto se construye cada día, por el contrario, se debe medir en términos de construir lo correcto cada día y lo correcto es aquello que agrega valor. Si no construimos valor, entonces estamos invirtiendo esfuerzo en desperdicio.





Para ello se utiliza la técnica **UVP** (User Value Proposition – Propuesta de Valor para el usuario) o (Unique Value Proposition – Propuesta única de valor) la cual está centrada en comprender las necesidades, deseos y problemas de los usuarios y en crear soluciones que satisfagan las necesidades de manera efectiva. Al principio nos imaginamos una hipótesis o un valor que tiene el producto/servicio a desarrollar → Visión: Resolver esa hipótesis única. Si en base a está desarrollamos un producto completo con muchas funcionalidades y recursos utilizados y lo comercializamos, estamos ante el riesgo de que este producto no sea algo que el mercado/usuarios estén demandando o que no cumpla las expectativas del usuario exigente. El MVP es el vehículo para probar esa hipótesis.

Mínimo Producto Viable (MVP)

Es un concepto de Lean Startup que enfatiza el impacto del aprendizaje en el desarrollo de nuevos productos. Se define como la versión de un nuevo producto que permite a un equipo recopilar la cantidad máxima de aprendizaje validado sobre los clientes con el menor esfuerzo. Este concepto le subyace la idea de que ver lo que la gente realmente hace con el producto es mucho más confiable que preguntarle a la gente qué harían. Una premisa clave detrás de la idea del MVP es que usted produce un producto real que puede ofrecer a los clientes y observar su comportamiento real con el producto o servicio.
No hay que confundirlo con MMF o MMP. La MMF (Característica Comerciable Mínima) o el MMP (Producto Comerciable Mínimo) a diferencia del MVP se enfocan en ganar, en cambio un MVP se enfoca en el aprendizaje.

Un MVP permite validar diferentes hipótesis:

- **Hipótesis del valor:** Prueba si el producto realmente está entregando valor a los clientes después de que comienzan a usarlo (ver si el producto es realmente el producto correcto).

- **Hipótesis de crecimiento:** Prueba cómo nuevos clientes descubrirán el Producto (ver si más personas desean usar el producto).

Para crear un MVP se sigue el siguiente roadmap:

1. Encontrar un nicho
2. Cread un roadmap realista
3. Investigar la competencia
4. Pre-vender el MVP
5. Testear las suposiciones
6. Asegurarse que el MVP resuelve el problema correcto
7. Focalizar en las funcionalidades principales

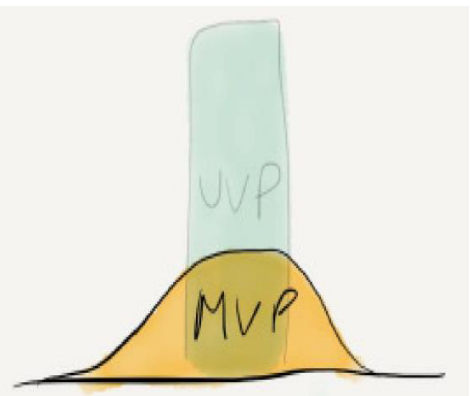
Design Thinking

O pensamiento de diseño, es una metodología para la creación de servicios digitales centrada en el cumplimiento de las expectativas del usuario mediante un proceso de mejora continua que advierte las verdaderas necesidades de este.
Para llevar a cabo su proceso, utiliza un esquema de pensamiento divergente y convergente. Con el primero se busca tener una visión amplia del espectro de soluciones que responden al problema inicial. El segundo consiste en reducir a la mejor idea el conjunto de posibles soluciones, con el objetivo de brindar aquella que es más adecuada para las necesidades del cliente. Su proceso es iterativo:

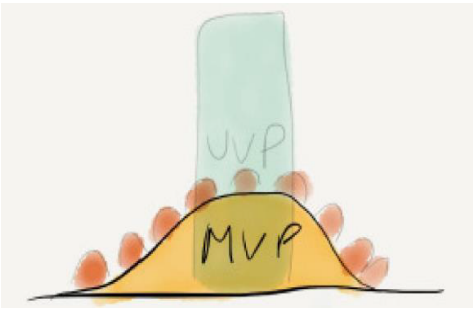
1. **Empatizar:** Implica comprender las necesidades del cliente mediante la utilización del razonamiento intuitivo.
2. **Definir:** Determinar lo valioso para el cliente a partir de lo recopilado durante la primera etapa, lo cual permitirá guiar el trabajo.
3. **Idear:** Articular muchas soluciones posibles. Esto se puede lograr en los equipos multifuncionales, donde cada miembro aporta una mirada diferente para generar muchas ideas para luego filtrar las más viables.
4. **Prototipar:** Consiste en materializar las ideas. Las ideas seleccionadas se transforman en prototipos que permitan visualizar la solución propuesta.
5. **Probar:** Es una de las etapas más importantes del proceso, ya que permite validar los prototipos con los usuarios implicados en la solución, obteniendo retroalimentación y permitiendo iterar.

Explicación MVP, MVF, MMF con el dinosaurio Agile

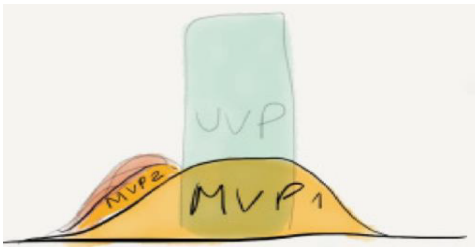
1) Idea del MVP: Trabajar con lo mínimo del producto que necesito obtener. Es decir, lo mínimo que debería tener el producto para que los (posibles) usuarios de este lo puedan evaluar y pueda indicar si el producto los atrae o no. Inicia la hipótesis de que el producto será único (UVP) y su objetivo es comprobar que la hipótesis del producto a construir funciona o si hay que hacer cambios.



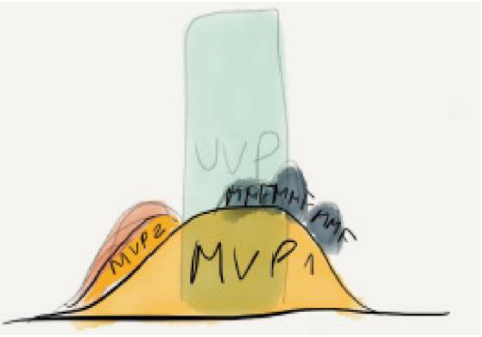
De esta forma la idea sería iniciar un circuito de aprendizaje, en donde, mediante la participación de los usuarios y la retroalimentación sobre si este cubre expectativas, sirve o si hay que hacer cambios en las características incluidas. No es vender el MVP, es validar la hipótesis



2) Tu MVP también es una hipótesis: Puede que nuestra hipótesis sirva o que cada usuario que compruebe nuestro MVP solicite funcionalidades diferentes.
Si por cada usuario que prueba el MVP se obtiene un feedback diferente, quiere decir que no está tan claro el mercado en el cual el producto funcionará (Por eso las desviaciones) y significa que deberemos seguir trabajando sobre la visión del producto para encontrarlo.

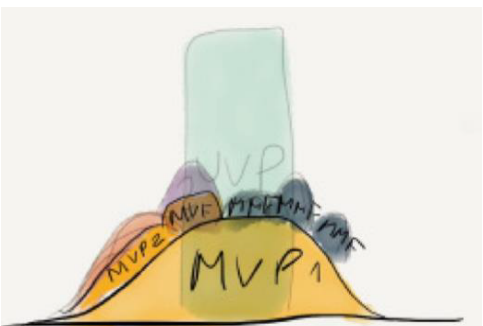


3) Caso contrario: Si ves cada vez más respuestas apuntando al MISMO X entonces tiene sentido revisar la hipótesis de Cliente/Problema/Solución. Básicamente, estás ejecutando un pivot. Está construyendo MVP2 centrado en la nueva hipótesis basada en el aprendizaje reciente de Desarrollo de clientes generado por el anterior MVP.



4) Ahora si es exitoso: Supongamos que MVP2 es exitoso y está viendo una atracción real de los primeros usuarios. Si desea aumentar el crecimiento y busca una penetración más profunda de sus primeros usuarios además de atraer nuevos clientes, algunos de ellos más allá de la multitud de usuarios pioneros. En función del feedback recopilado y la investigación de su gestión de productos, tiene un par de áreas que potencialmente pueden traer este crecimiento. Surge el concepto de:
MMF (Minimun Marketable Feature – Característica mínima comercializable): Está enfocado en definir elementos mínimos que deben estar presentes en un producto o servicio para que sea comercializable. Su objetivo es aportar valor. Supone una alta certeza de que existe valor en esta área y que sabemos cuál debe ser el producto para proporcionar este valor. La razón para dividir una característica grande en MMF más pequeños es principalmente el tiempo de comercialización (Time to market) y la capacidad de aportar valor en muchas áreas. Su objetivo es asegurarnos que este primer MMF sirve o no. Diferenciando en que ahora hablamos de una Feature, lo cual es más pequeño que el MVP, siendo lo mínimo que puedo sacar a la venta.

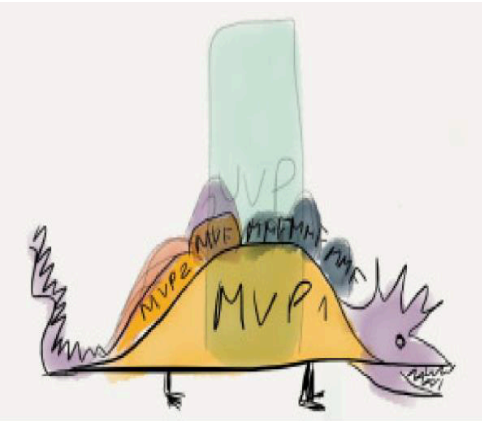
5) Unificando: Si se desea esperar para ver si el primer MMF apesta ... entonces está de vuelta en la tierra de la hipótesis. Ahora tu hipótesis se centra en una característica en lugar del producto. Tienes un área con alto potencial pero también alta incertidumbre. La forma de afrontarlo es crear una función "pionera". Surge el concepto de MVF:



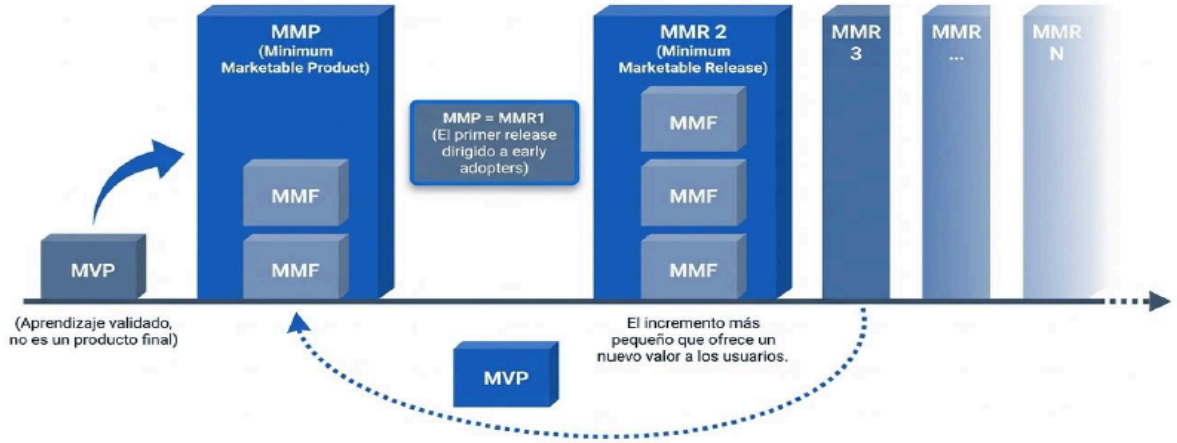
MVF (Minimun Viable Feature – Característica mínima viable):
Surge al combinar el MVP con la MMF, se trata de la característica mínima que se puede construir e implementar rápidamente, utilizando recursos mínimos para probar la utilidad de esta.

Permite validar cual es la característica que hace la diferencia y la que será por la cual compren el producto.
Objetivo: Buscamos que esta Feature ofrezca un valor agregado, en donde la hipótesis se centra en una característica en vez de un producto.
Si es exitosa se pueden desarrollar más MMF en esta área para tomar ventaja (ya que está comprobada), siendo una versión mini del MVP.

6) Modelo completo: El producto se cultiva en mercados inciertos al intentar varios MVP. Cuando se logra ajustar el producto en el mercado de productos se combinan MMF y MVF según el nivel de incertidumbre del negocio / requisitos en las áreas en las que se está enfocando. Si bien los MVP / MMF / MVF son atómicos desde una perspectiva empresarial (no puede implementar y aprender de algo más pequeño) pueden ser bastante grandes desde la perspectiva de la implementación. El dinosaurio carpaccio se obtiene cortando cada una de esas piezas en pequeñas porciones destinadas a reducir el riesgo de ejecución / tecnología (normalmente se denominan User Stories). Esas porciones más pequeñas pueden tener un valor comercial tangible o no. Para otros puede ser importante proporcionar una temprana retroalimentación de decisiones a lo largo del camino.



MRF (Minimum Release Feature – Características mínimas del release): Trata del reléase del producto que tiene el conjunto de características más pequeño posible. En donde el incremento más pequeño, ofrece un valor nuevo a los usuarios y satisface sus necesidades actuales.



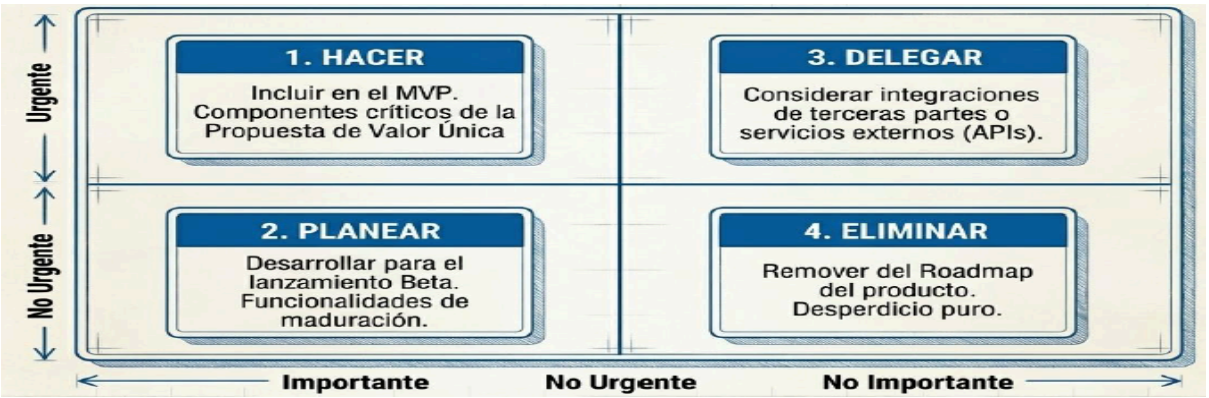
Además, se agrega un nuevo concepto:
Minimum Lovable Product (MLP - Producto mínimo adorable): Surge luego del MMP comercializable, ya que lo viable ya no es suficiente y en mercados maduros dominados por competidores bien financiados, un MVP tradicional puede parecer un intento de lucro a corto plazo. Va más allá de la viabilidad:

- Reputación de marca: Demuestra a los usuarios que te importan sus necesidades
- Diferenciación: Destaca donde lo funcional es un estándar mínimo
- Feedback superior: Crea una conexión emocional temprana para retener usuarios leales
- Los 10 bloques de construcción de la adorabilidad son:

- Halo
- Motivación, Diversión (piso de inspiración, perseguir una vida significativa)
- Confianza en uno mismo, Confianza en el sistema, Escala, Sostenibilidad (piso de crecimiento, me ayuda a seguir creciendo)
- Esperanza, Satisfacción, Cuidado (utilidad, resuelve mis problemas)

Comparación entre MVP, MMF, MVF, MRF y MLP

	MVP (Producto mínimo viable)	**MMF (Característica mínima comercializable) (o MMP, producto mínimo comercializable)**	**MVF (Característica mínima viable)**	**MRF (Características mínimas del release)**	**MLP (Producto mínimo adorable)**
Enfoque:	Entender necesidades. Validar conceptos rápidos y barato	Amplio alcance / atender a grandes bases de usuarios	Validar una funcionalidad específica de manera rápida	Entregar un conjunto mínimo de funcionalidades listo para ser utilizado	Diseñar para el deleite / conexión emocional
Foco clave:	Investigación, código base, pruebas de usuario	Estabilidad técnica, lanzamiento al mercado	Funcionalidad principal, validación con usuarios	Calidad, estabilidad y funcionalidades necesarias para el lanzamiento	UI avanzada, impacto emocional, reducción de fricción
Resultado:	Informar el alcance del producto	Atracción de mercado masivo	Validar una funcionalidad y obtener feedback	Producto listo para ser lanzado	Lealtad temprana y retención





Ciclo de Feedback o Aprendizaje (Lean UX): El éxito no es entregar un producto, el éxito se trata de entregar un producto (o característica de producto) que el cliente usará.
La forma de hacerlo, es alinear los esfuerzos continuamente hacia las necesidades reales de los clientes. The Build-Experiment-Learn Feedback Loop permite descubrir las necesidades del cliente y alinearlas metodológicamente. Es un ciclo entre: Idea, construir, producto, experimentar, datos y aprender.

Framework Scrum (PPT 8)

Scrum es un marco de trabajo ligero que ayuda a las personas, equipos y organizaciones a generar valor a través de soluciones adaptables para problemas complejos. En pocas palabras, Scrum requiere un Scrum Master para fomentar un entorno donde:

1. Un propietario del producto (Product Owner) ordena el trabajo de un problema complejo en un Product Backlog.
2. El equipo de Scrum convierte una selección del trabajo en un Incremento de valor durante un Sprint.
3. El equipo de Scrum y sus partes interesadas (stakeholders) inspeccionan los resultados y realizan los ajustes necesarios para el próximo Sprint.
4. Repetir

En el marco se pueden emplear diversos procesos, técnicas y métodos. Scrum envuelve las prácticas existentes o las hace innecesarias. Scrum hace visible la eficacia relativa de la gestión actual, el entorno y las técnicas de trabajo, de modo que se pueden realizar mejoras. Scrum se basa en el empirismo y el pensamiento Lean. El empirismo afirma que el conocimiento proviene de la experiencia y la toma de decisiones basadas en lo que se observa. El pensamiento Lean reduce los desperdicios y se centra en lo esencial
¿Por qué decimos que Scrum se apoya en los pilares del empirismo?:

- **Transparencia:** Tanto el proceso como el trabajo emergente deberá ser visible para cualquier involucrado. En Scrum, las decisiones importantes se basan en el estado percibido de sus tres artefactos formales → Sin transparencia conducirán a decisiones que disminuyen el valor y aumentan el riesgo. “La transparencia permite la inspección, ya que la inspección sin transparencia genera engaños y desperdicios.”
- **Inspección:** Los artefactos y el progreso hacia objetivos acordados deben ser inspeccionados con frecuencia y diligentemente para detectar varianzas/problemas potencialmente indeseables → Scrum plantea eventos para llevar la cadencia. “´La

inspección permite la Adaptación, ya que la inspección sin adaptación se considera inútil. Los eventos de Scrum están diseñados para provocar cambios."

- **Adaptación:** Ante cualquier desviación en un aspecto del proceso o si el mismo es inaceptable, el proceso que se está aplicando o los materiales que se producen deben ajustarse. El ajuste se realizará lo antes posible para minimizar la desviación adicional. Se vuelve difícil cuando las personas involucradas no están empoderadas o no poseen capacidad para autogestionarse. El equipo de Scrum deberá adaptarse al momento que aprenda algo nuevo mediante la inspección.

Valores de scrum: El sistema operativo de la confianza: Cinco valores principales: Compromiso, Enfoque, Apertura, Respeto y Coraje. El equipo de Scrum se compromete a lograr sus objetivos y apoyarse mutuamente.

Scrum Team

Dentro de un equipo de Scrum, no hay sub-equipos ni jerarquías. Es una unidad cohesionada de profesionales enfocada en un objetivo a la vez, el objetivo del Producto.
Scrum Master: Es el responsable de establecer Scrum tal como se define en la Guía de Scrum, ayuda a todos a comprender la teoría y la práctica de Scrum, tanto dentro del equipo como en toda la organización. Es el responsable de la efectividad del Scrum Team, permitiéndole a todo el equipo que mejore sus prácticas dentro de este marco – Son verdaderos líderes que sirven al equipo Scrum y a toda la organización.

Product Owner – Dueño del producto: Es el responsable de maximizar el valor del producto resultante del trabajo del equipo de Scrum. También es responsable de la gestión eficaz del Product Backlog – Representa al equipo.

Developers – Desarrolladores: Personas del equipo Scrum, comprometidas a crear cualquier aspecto de un incremento útil (funcional) en cada Sprint.

Es un equipo pequeño de profesionales multifuncionales que está enfocado en un objetivo a la vez (Producto). En donde c/u de los miembros tienen todas las habilidades necesarias para crear valor en cada Sprint a su vez son autogestionados por lo que internamente deciden quién hace qué, cuándo y cómo. También es demasiado grande como para completar un trabajo significativo dentro de un Sprint (10 o menos personas). El equipo es el responsable de todas las actividades relacionadas con los productos, desde la colaboración, verificación, mantenimiento, operación, experimentación, investigación y desarrollo, y cualquier otra cosa que pueda ser necesaria.

En la guía extendida (2026) se agregaron 2 nuevos roles:
Stakeholder: Son las entidades interesadas o afectadas por el producto (clientes, usuarios, patrocinadores, legisladores, etc)

Supporter: Stakeholder específico en la que su responsabilidad principal es la coalición poderosa que elimina impedimentos organizacionales y fomenta un clima para la autogestión

Además se agrega el ciclo de empirismo **OODA:** Observar - Orientar - Decidir - Actuar. Para responder rápidamente a los cambios del mercado y a la incertidumbre.
Además que puede haber una reunión más, refinamiento del PB (actividad continua)

Otro concepto de esta guía extendida es la diferencia entre output y outcome:

- **Increment (Output):** Salida del sprint (SW). Incremento utilizable. ¿Construimos el producto correctamente?
- **Product (Outcome):** Resultado. Se mide en término del valor del negocio que el output generó. Evidencia observable de valor. ¿Construimos el producto correcto?

4 eventos de Scrum

Son cuatro eventos formales para la inspección y adaptación dentro de un evento contenedor → Sprint.
Sprint: Son eventos de longitud fija de un mes o menos para crear consistencia. Un nuevo Sprint comienza inmediatamente después de la conclusión del Sprint anterior. Todo el trabajo necesario para alcanzar el objetivo del producto, incluyendo la Planificación (Sprint Planning), Daily Scrums, Revisión del Sprint (Sprint Review) y la Retrospectiva (Sprint Retrospective), ocurren dentro del Sprints. Durante el Sprint:

- No se hacen cambios que pongan en peligro el Objetivo Sprint.
- La calidad no disminuye.
- El trabajo pendiente del producto se refina según sea necesario.
- El alcance se puede clarificar y renegociar con el Propietario del Producto a medida que se aprende más.
- Permiten la previsibilidad al garantizar la inspección y adaptación del progreso hacia un objetivo del producto.
- Formas de Pronosticar el progreso: Gráficos de Burn-Downs, Burn-Ups o flujos acumulativos.
- Un Sprint podría ser cancelado si el objetivo de este se vuelve obsoleto. Solo el PO tiene la autoridad para cancelarlo.

1) Sprint Planning – Planificación de Sprint: Inicia el sprint estableciendo el trabajo que se realizará para el mismo. Este plan resultante es creado por el trabajo colaborativo de todo el equipo de Scrum.
El PO se asegura que los asistentes estén preparados para discutir los elementos de trabajo pendiente de producto más importante y cómo se asignan al objetivo del producto. Abarca temas como: ¿Por qué este Sprint es valioso?, ¿Qué se puede hacer este Sprint?, ¿Cómo se realizará el trabajo elegido?

2) Daily Scrum – Scrum diario: Su propósito es inspeccionar el progreso hacia el objetivo sprint (sprint goal) y adaptar el Sprint Backlog según sea necesario, ajustando el próximo trabajo planeado. Suele durar 15 minutos como máximo para los desarrolladores del equipo Scrum. Estos mejoran la comunicación, identifican impedimentos, promueven una rápida para la toma de decisiones, y en consecuencia, eliminan la necesidad de otras reuniones. Preguntas rápidas cómo que hiciste desde el ultimo daily hasta hoy / que vas a hacer hoy.

3) Sprint Review – Revisión del Sprint: Su propósito es inspeccionar el resultado del Sprint y determinar futuras adaptaciones. Es donde el equipo presenta los resultados de su trabajo a las partes interesadas clave y se discute el progreso hacia el objetivo del producto. Aquí es donde el equipo y las partes interesadas revisan lo logrado en la Sprint y lo que ha cambiado en su entorno. En base a esto, se decidirá qué hacer a continuación. Todo trabajo pendiente del producto se puede ajustar para satisfacer nuevas oportunidades.

4) Sprint Retrospective – Retrospectiva del Sprint: Su propósito es planificar formas de aumentar la calidad y la eficacia. En donde el equipo inspeccionará como fue el último Sprint respecto a individuos, interacciones, procesos, herramientas y su DoD.
También las suposiciones que los desviaron son identificadas y se exploran sus orígenes. El equipo analiza qué fue bien durante el Sprint, qué problemas encontró y cómo esos problemas fueron (o no) resueltos.
Aquí el equipo identifica los cambios más útiles para mejorar su eficacia. Las mejoras más impactantes se abordan lo antes posible. Incluso se pueden agregar al Sprint Backlog para el próximo sprint.
Este evento concluye el Sprint, tiene una duración de hasta máximo de 3 horas para un Sprint de un mes

Timebox (caja de tiempo): Esto existe para que no se pierda más tiempo del necesario en las distintas tareas y para aprender a negociar en términos del alcance del producto y no del tiempo o los costos (relacionado con la triple restricción).
Si no se llega al final de un sprint, en vez de extender el tiempo, se entrega menos.
De esta manera, nos volvemos más confiables, con un ritmo de trabajo sostenible. Todas las ceremonias son Timebox. El refinamiento del Product Backlog al ser una actividad continua, no tiene definido un tiempo exacto, sino que el tiempo se expresa en términos porcentuales sobre la definición del Sprint. Estos intervalos pueden ser menos pero no más.

- **Sprint:** 1 mes o menos
- **Sprint planning:** 8 horas máximo para un sprint de 1 mes
- **Daily meeting:** 15 minutos
- **Sprint review:** 4 horas máximo para un sprint de 1 mes
- **Sprint retrospective:** 3 horas máximo para un sprint de 1 mes

3 artefactos de Scrum

Representan trabajo o valor. Están diseñados para maximizar la transparencia de la información clave. Cada artefacto contiene un compromiso para garantizar que proporciona información que mejora la transparencia y el enfoque con el que se puede medir el progreso:

- Product backlog → Para medir el product goal (objetivo estratégico a mediano plazo)
- Sprint backlog → Sprint goal
- Output → Outcome

1) Pila del Producto – Product Backlog: Lista emergente y ordenada de lo que se necesita para mejorar el producto → Es el trabajo pendiente. Está lista puede ser hecha por el equipo dentro de un Sprint y se consideran listos para su selección en una Planning. →

Para ello deben cumplir con el Definition of Ready. Adquieren el grado de transparencia luego de la refinación.

- **Refinación de backlog:** Se descomponen y definen aún más los elementos de trabajo pendiente del producto en artículos más pequeños y precisos.

Dentro de un PB nos encontramos cualquier forma de productos, NO tareas, si no características, que el producto deberá tener, es decir: US, Epicas, Temas, Spikes, Restricciones, Tech Stories, Deuda Tecnica y Defectos (Post Iteraciones)
Product Goal: Describe un estado futuro del producto que puede servir como objetivo para el equipo contra el cual planificar. Forma parte del PB, en donde el resto de trabajo pendiente del producto surge para definir "qué" cumplirá el objetivo del producto.

2) Pila del Sprint – Sprint Backlog: El trabajo pendiente de Sprint se compone del objetivo sprint (Por qué), el conjunto de elementos de trabajo pendiente de producto seleccionado para el sprint (qué), así como un plan accionable para entregar el incremento (cómo). El trabajo pendiente de Sprint es un plan por y para desarrolladores. Siendo una imagen visible y en tiempo real donde estos deberán realizar durante el Sprint para cumplir el Objetivo Sprint.

- **El Sprint Backlog (SB):** Es actualizado a lo largo del Sprint a medida que se aprende más. Debiendo tener los suficientes detalles como para poder inspeccionar su progreso en la Daily Scrum.

Objetivo Sprint – Sprint Goal: Es el único objetivo para la Sprint. Es más, un compromiso de los desarrolladores, proporcionando flexibilidad en términos de trabajo exacto necesario para logarlo. Esté, crea coherencia y enfoque, animando a el trabajo en conjunto. Se crea durante la Planning, y es agregado al trabajo pendiente de Sprint.

3) Incremento – Increment: Cada incremento es aditivo a todos los Incrementos anteriores y verificado a fondo, asegurando que todos funcionen juntos, de esta forma proporciona el valor, el incremento debe ser utilizable. Se pueden crear varios incrementos dentro de una Sprint, en donde la suma de estos es presentada en la Revisión Sprint apoyándose en el empirismo. El trabajo no puede ser considerado parte de un incremento a menos que cumpla con el DoD (Definition of Done).
Definition of Done – DoD – Definición de Hecho: Es una descripción formal del estado del Incremento cuando cumple con las medidas de calidad requeridas para el producto.

Capacidad del equipo en un sprint: Métrica utilizada por Scrum, siendo la única estimable y utilizada para ver cuanto compromiso de trabajo puede asumir el equipo en un determinado sprint. Lo vemos reflejado en la Planning, donde determinamos la capacidad del equipo, teniendo en cuenta está, es contrastada en base a las US del PB que podrán pasar por el Sprint Backlog y hasta donde nos podemos comprometer en un determinado Sprint.
Se estima porque estamos comenzando el sprint y observamos cuantas horas reales podremos dedicarle al trabajo en el Sprint. La capacidad se estima en horas ideales/Story Points (si es un equipo maduro). Por lo que la primera variable de definición es cuánto va a durar el sprint, la segunda será el cálculo de capacidad. Finalmente se acordará el objetivo en base a esas tres variables se indica qué cosas van a empezar a pasarse desde el Product Backlog hasta el Sprint Backlog.

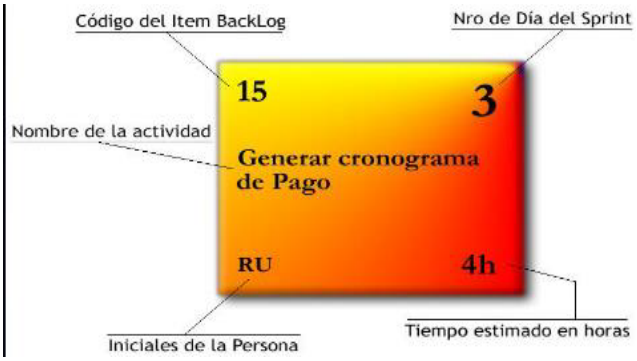
Analizando sus niveles de planificación:

Nivel	Horizonte	Quién	Foco	Entregable
Portfolio	1 año o más	Stakeholders y Product Owners	Administración de un Portfolio de Producto	Backlog de Portfolio
Producto	Arriba de varios meses o más	Product Owner y Stakeholder	Visión y evolución del producto a través del tiempo	Visión de Producto, roadmap y características
Release	3 (o menos) a 9 meses	Equipo Scrum entero y Stakeholders	Balancear continuamente el valor del cliente y la caldad global con las restricciones de alcance, cronograma y presupuesto	Plan de reléase
Iteración	Cada iteración (de 1 semana a 1 mes)	Equipo Scrum entero	Que aspectos entregar en el siguiente Sprint	Objetivo del Sprint y Sprint Backlog
Día	Diaria	Scrum Master y Equipo de desarrollo	Cómo completar lo comprometido	Inspección del progreso

Herramientas de scrum

1) Tarjeta de tarea: Es una tarjeta que cuenta con 5 partes:

1) Código del Ítem Backlog: es un identificador del ítem en el Product Backlog.
2) Nombre de la actividad: suele ser una frase verbal que define qué es lo que hay que hacer.
3) Iniciales del responsable de realizar la tarea.
4) Número de día del sprint.
5) Estimación del tiempo para finalizar la tarea



2) Tablero: Utilizado por el Equipo de Desarrollo en el cual se colocan las tarjetas de tareas a realizar durante el sprint. Se encuentra dividido en 5 secciones:

1. **Story:** El nombre de la historia.
2. **To Do:** Aquí se encuentran las tareas por hacer en el Sprint.
3. **Work In Process:** Aquí se encuentran las tareas que se están realizando.
4. **To Verify:** Aquí se encuentran las tareas finalizadas que deben verificarse.
5. **Done:** Aquí se encuentran las tareas terminadas: finalizadas y verificadas.

Lo ideal es que el nivel de granularidad de las tareas sea lo más fino posible, de tal forma que se detalle profundamente el avance del sprint, teniendo varias tareas en cada sección del tablero. Scrum recomienda la estimación mediante Story Points, ya que son estimaciones sobre el producto y no sobre el trabajo.

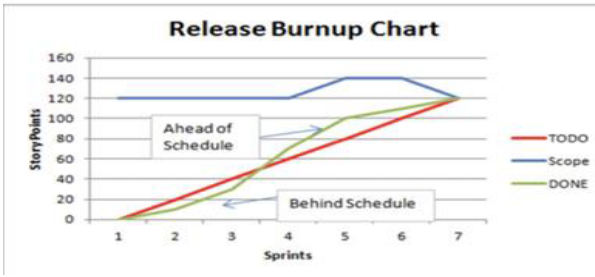
3) Gráficos del backlog: Gráficos que brindan información acerca del progreso de un Sprint, de una release o del producto. El backlog de trabajo es la cantidad de trabajo que queda por ser realizado (en horas), mientras que la tendencia del backlog compara esta cantidad con el tiempo (medido en días).

Sprint Burndown Charts: Son gráficos que permiten la visualización del trabajo. En el eje “y” se muestran los puntos



de historia restante y en el eje “x” se colocan los días. No conviene realizarlo con las horas de trabajo, ya que no es representativo del esfuerzo o logro obtenido. Sirven para calcular velocidad y se descartan al final del Sprint.

Sprint Burnup Chart: Son gráficos que no se descartan al final de un sprint, permiten realizar un seguimiento de como se va trabajando en el producto a lo largo de los sprints

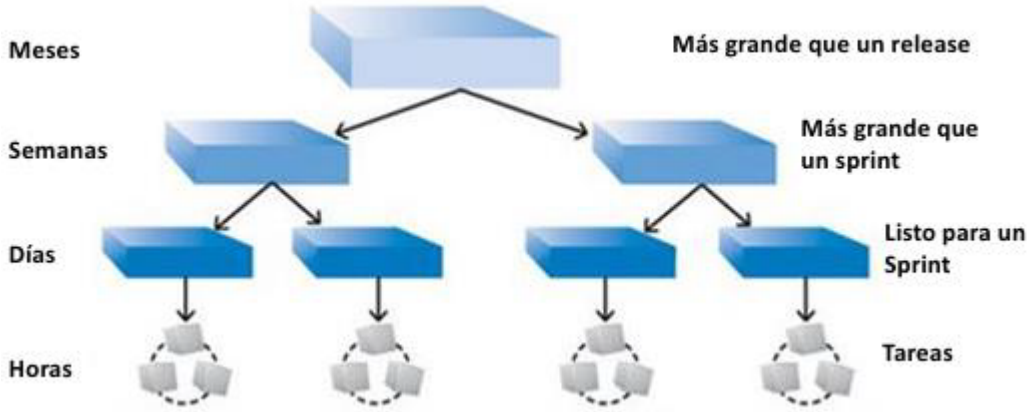


Planificaciones en SCRUM

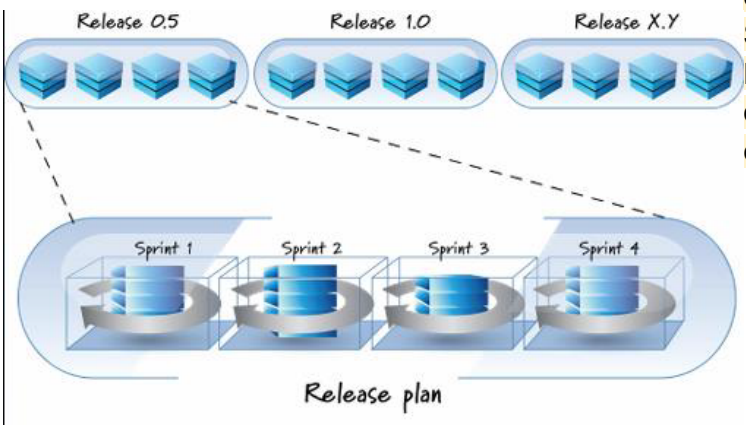
Depende puramente del nivel de granularidad que se está manejando. Pero existen ciertas recomendaciones para estimar.

- **Portafolio:** Estimación en talles de remeras.
- **Product Backlog:** Estimación mediante Story Points, también se puede con talles de remera. Pero mientras mejor priorizados estén los ítems del PB, más fácil será su priorización.
- **Sprint Backlog:** Se utilizan historias con DoR, estimadas en Story Points. Luego el equipo es quien decide si dividirá las historias en tareas o no. Si lo hacen, las tareas se estiman en horas ideales.

Planificación del Release: La salida de la planificación del release son las características que quiero desarrollar (en un rango), cuantos sprints serán necesarios para terminar el reléase, la capacidad estimada del equipo, objetivos y características de cada sprint. En función de eso se realiza una planificación, al ocurrir una variación, se deberá replanificar. Distintos niveles de granularidad: Algo más grande que una US, no entrará en una Sprint. Si son características muy grandes, no entrará siquiera en un Release, se deberá repartir en varias releases.



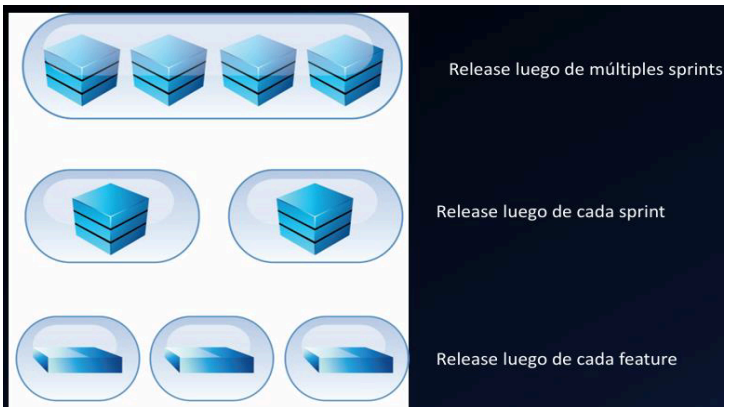
Las tareas NO están en el Product Backlog. Dentro del Sprint Backlog. tenemos Product Backlog de granularidad diferentes niveles en donde las



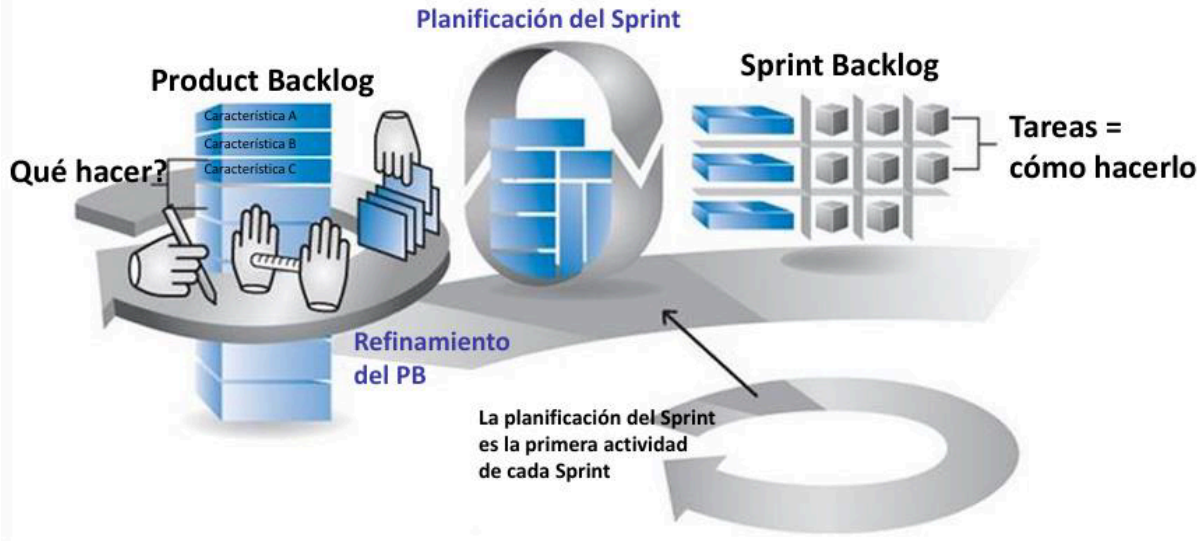
releases se separarán en Sprints.

En la imagen observamos que la velocidad promedio es de 20 Story Points, por lo que se sumarán características hasta alcanzar ese valor y se le asigna un Sprint. En los primeros sprint, no sabremos bien la velocidad del equipo, hasta que vayamos logrando experiencia en las sucesivas iteraciones.

Cadencia de los Sprints: Existen varias formas de armar las releases, algunas empresas se liberan luego de terminar varios sprint, en otras hay una reléase por sprint y en otras luego de cada feature.



Planificación de Iteración – Sprint planning: Este proceso debe tener como resultado la configuración del tablero, teniendo como entrada el objetivo del sprint, su duración, la capacidad estimada del equipo. En donde el equipo dividirá en tareas cada US, estimándolas en horas. El proceso de planificación del sprint termina con el Sprint Backlog configurado.



Unidad 3: Gestión del Software como producto

Al pensar en software pensamos en un conjunto de: Programas, Procedimientos, Reglas, Documentación y Datos. En donde la información tiene que ser:

- Estructurada con propiedades lógicas y funcionales.
- Creada y mantenida en varias formas y representaciones.
- Confeccionada para ser procesada por computadora en su estado más desarrollado.

Cambios en el Software: Estos deben su origen a:

- Cambios del negocio y nuevos requerimientos.
- Soporte de cambios de productos asociados.
- Reorganización de las prioridades de la empresa por crecimiento.
- Cambios en el presupuesto.
- Defectos encontrados a corregir.
- Oportunidades de mejora.
- Solo no tiene cambios si nadie usa el software.

SCM (PPT 2)

Gestión de la configuración de software (Software Configuration Management - SCM): Es una disciplina que aplica dirección y monitoreo administrativo y técnico a: identificar y documentar las características funcionales y técnicas de los ítems de configuración, controlar los cambios de esas características, registrar y reportar los cambios y su estado de implementación y verificar correspondencia con los requerimientos

Es considerada una disciplina de soporte ya que ocurre transversalmente a lo largo de todo el proyecto, pero trasciende para dar soporte al producto en todo su ciclo de vida. Pero también tiene aplicación en diferentes disciplinas como:

- Control de calidad de proceso.
- Control de calidad de producto.
- Prueba de software.

Su propósito es resolver problemas de diferente índole, a través del establecimiento y el mantenimiento de la integridad del producto de software a lo largo de todo su ciclo de vida.

- Estos problemas pueden ser pérdida de componentes o cambios, cambios no validados o con superposición, etc.

Su propósito es establecer y mantener la integridad de los productos de software a lo largo de su ciclo de vida. Esto implica identificar la configuración en un momento dado, controlar sistemáticamente sus cambios y mantener su integridad y origen.

- La integridad es el medio por el cual podemos garantizar que el producto a entregar tiene la calidad correspondiente. Y nos garantiza un nivel mínimo de confiabilidad.
- El problema de la calidad es que es subjetiva y se suma a que el software es intangible, por lo tanto, es muy difícil medir si la calidad que responde al cumplimiento de las expectativas del cliente realmente se verifica.

- La idea de la integridad del producto es hacer explícita las características o expectativas que tiene el cliente sobre el producto de software.

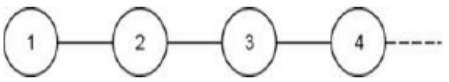
Decimos que se mantiene la integridad de un producto de software cuando:

- **Satisface las necesidades de usuario:** Hace lo que el usuario espera que haga
- **Permite la rastreabilidad durante todo su ciclo de vida:** Existen vínculos o conexiones entre los ítems de configuración (los cuales deben ser definidos desde un primer momento del ciclo de vida del producto), que permiten analizar en donde impactará un cambio en un ítem de configuración. De esta forma, se puede determinar cómo impactará cada cambio de requerimientos a través de la trazabilidad. A mayor cantidad de vínculos → mayor información → mayor trazabilidad → mayor costo (analizar la relación costo-beneficio);
- **Satisface criterios de performance y RNF**
- **Cumple con expectativas de costo:** Este apartado incluye la satisfacción del equipo de proyecto. No solo el cliente debe estar satisfecho, sino también el desarrollo del producto debe ser redituable.

Conceptos generales

Ítem de configuración (IC): Son aquellos artefactos que forman parte del producto o proyecto, y pueden ser almacenados en un repositorio, sin importar su extensión/tipo. Pueden sufrir cambios, y se desea conocer su estado y evolución a lo largo del ciclo de vida (ya sea del producto o proyecto, dependiendo del artefacto). Es decir, es cualquier aspecto asociado al producto de software (requerimientos, diseño, código, datos de prueba, documentos, etc.) que es necesario mantener. Son todos aquellos elementos que componen toda la información producida como parte del proceso de ingeniería de software, como ser programas de computadora (código fuente y ejecutables), documentos que describen los programas (documentos técnicos o de usuario), datos (de programa o externos). Son digitalizables y almacenables. EJ: íconos, manual de usuario, ERS, etc.

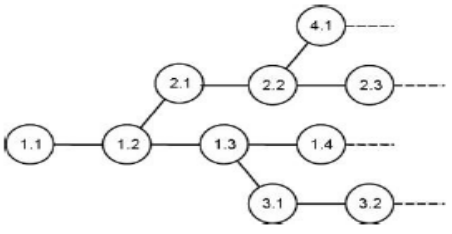
Versión: Instancia de un ítem de configuración que difiere de otras instancias de este ítem. Las versiones se identifican unívocamente. Controlar una versión refiere a la evolución de un único ítem de configuración.
"La gestión de configuración permite a un usuario especificar configuraciones alternativas del sistema de software mediante la selección de las versiones adecuadas"; esto se puede gestionar asociando atributos a cada versión (que pueden ser datos sencillos como un nro. de versión asociado a cada objeto. Cada versión de software es una colección de elementos de configuración (ECS), como ser código fuente, documentos, datos).
La versión es un punto particular en el tiempo de ese ítem de configuración (es un estado). Una versión se define, desde el punto de vista de la evolución, como la forma particular de un artefacto en un instante o contexto dado. La evolución puede representarse gráficamente en forma de grafo.



Evolución lineal de un ítem de configuración

Control de versiones: Es la evolución de un único ítem de configuración (IC), o de cada IC por separado.

Variante: Versión de un ítem de configuración (o de la configuración) que evoluciona por separado.



Las variantes representan configuraciones alternativas. Las variantes de un ítem de configuración permiten satisfacer requerimientos en conflicto, como por ejemplo la compatibilidad del producto con diferente hardware o sistemas operativos. Si la diferencia que existe entre las instancias de un mismo ítem de configuración es muy pequeña, no se denomina versión, sino que se la conoce como variante. Está ligado a trabajar con Branches, donde el producto adopta distintas configuraciones, dependiendo de múltiples factores (problemas, SO donde se ejecutará, etc.).

Configuración: Conjunto de todos los ítems de configuración con su versión específica. Define una foto de los ítems de configuración en un momento de tiempo determinado. Una configuración es el conjunto de todos los componentes fuentes que son compilados, sus documentos y la información de la estructura que definen una versión del producto a entregar. La configuración de un software es la sumatoria de todos los ítems de configuración que tiene en un momento determinado, equivale a una instantánea o una foto de todos los ítems de configuración con su versión en un momento del tiempo.

Repositorio: Es un contenedor de ítems de configuración, se encarga de mantener la historia de cada ítem con sus atributos y relaciones, además es usado para hacer evaluaciones de impacto de los cambios propuestos. Puede ser una o varias bases de datos. Tiene una estructura para mantener el orden y la integridad. El que tenga una estructura ayuda a la seguridad, los controles de acceso, las políticas de Backup, etc.

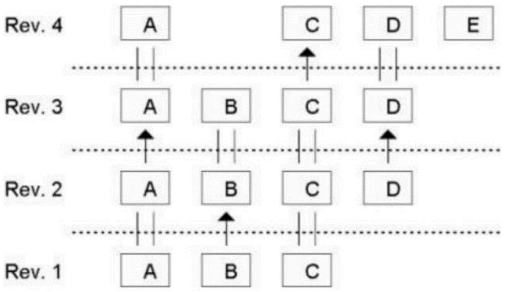
- **Repositorios centralizados:** Un solo servidor contiene todos los archivos con sus versiones. Los administradores tienen un mayor control sobre el repositorio. Tiene como desventaja qué si falla el servidor, todo lo positivo se cae
- **Repositorios descentralizados:** Cada cliente tiene una copia exactamente igual del repositorio completo. Tiene como ventaja que se resuelve el problema de los servidores centralizados, debido a que si el servidor falla sólo es cuestión de “copiar y pegar”, además posibilita otros workflows no disponibles en el modelo centralizado. La desventaja que podemos mencionar es que es más complicado llevar un control sobre el mismo.

Release: Entrega de un sistema que se libera para su uso a los clientes u otros usuarios de la organización.

Rama (Branch): Es un conjunto de ítems de configuración con sus correspondientes versiones, que permiten bifurcar el desarrollo de un software, ya sea por experimentación, resolución de errores en el desarrollo, desarrollar un sw para diferentes plataformas, etc.
Al realizar la bifurcación de la línea donde están los IC, normalmente esta es la LB, pero no es considerada buena práctica trabajar de accionar sobre esos IC situados en la rama principal o Master. Se utilizan bifurcaciones para no trabajar sobre la Master, estas pueden ser descartadas o integradas y son utilizadas para experimentación entre otras. Al aceptar una bifurcación con la rama Master, estamos en presencia de un Merge, ante un conflicto se resuelve mediante Diff. Todas las ramas deberían eventualmente integrarse a la principal o ser descartadas.

Integración de ramas (Merge): Se da cuando se integran dos ramas, para fusionar la configuración de los IC con sus correspondientes versiones, en cada una de ellas.

Una buena práctica es mantener la rama principal como la versión estable, y fusionar los cambios hacia ella. En caso de no integrarse a la Main, las ramas deberían descartarse.



Línea base (LB): Puede contener en un momento de tiempo un solo IC o un conjunto de ítems. Está conformada por IC que se toman como referencia, los cuales han sido considerados estables en base a los niveles de revisión y aprobación.
La LB debe tener una identificación unívoca (se hace con tags), debe tener un nombre, una versión, una identificación única en un momento y tiene que indicar claramente cuáles son los IC con su estado que forman parte de esa determinada LB. Por lo que:

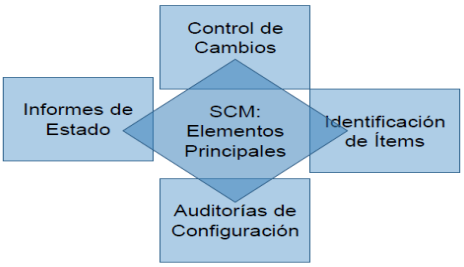
- Sirve como base para desarrollos posteriores y puede cambiarse sólo a través de un procedimiento formal de control de cambios.
- Permite ir atrás en el tiempo y reproducir el entorno de desarrollo en un momento dado del proyecto.
- Se acuerdan parámetros para determinar qué se considera o qué debe cumplir los ítems de configuración para considerarse como línea base
- Aquello que no forme parte de está, podrá ser modificado con total libertad.
- **Utilidad:** Funciona como un punto de referencia, útil para realizar rollbacks o saber que se pone en producción. Permite saber cuál era la última situación estable en un momento de tiempo y cómo se fue evolucionando. Se suele poner mínimamente uno después de cada etapa.
- **Tipos:**
    - **De especificación:** Son de requerimientos, diseño. No poseen código, poseen información de la ingeniería del producto.
    - **Operacionales:** LB que ya tienen código, han pasado por un control de calidad definido previamente. La primera línea base operacional corresponde con la primera Release

Planificación de la gestión de configuración de software: Este plan se debe confeccionar al inicio de un proyecto, se deben definir los documentos que se administrarán (todos los productos del proceso deben administrarse) y existen diferentes estándares donde se expresa cómo planificar:

- Reglas de nombrado de los ítems de configuración;
- Herramientas para utilizar en SCM;
- Roles e integrantes del Comité de Control de Cambios;
- Procedimientos formales de cambios;
- Procesos de auditoría;
- Estructura del repositorio;
- SCM para software externo (opcional);
- Cómo se hará el control de cambios;
- Registros que deben mantenerse;
- Tipos de documentos.

Actividades fundamentales del SCM

Estas actividades/elementos son las cosas que contienen las secciones de un plan de gestión de configuración. Recordemos que la gestión de configuración también se debe planificar.
Todas las actividades se realizan en ambas metodologías, menos las auditorías, ya que el concepto de ser auditado y controlado por alguien externo, no es compatible con una metodología ágil pura.



Identificación de ítems de configuración

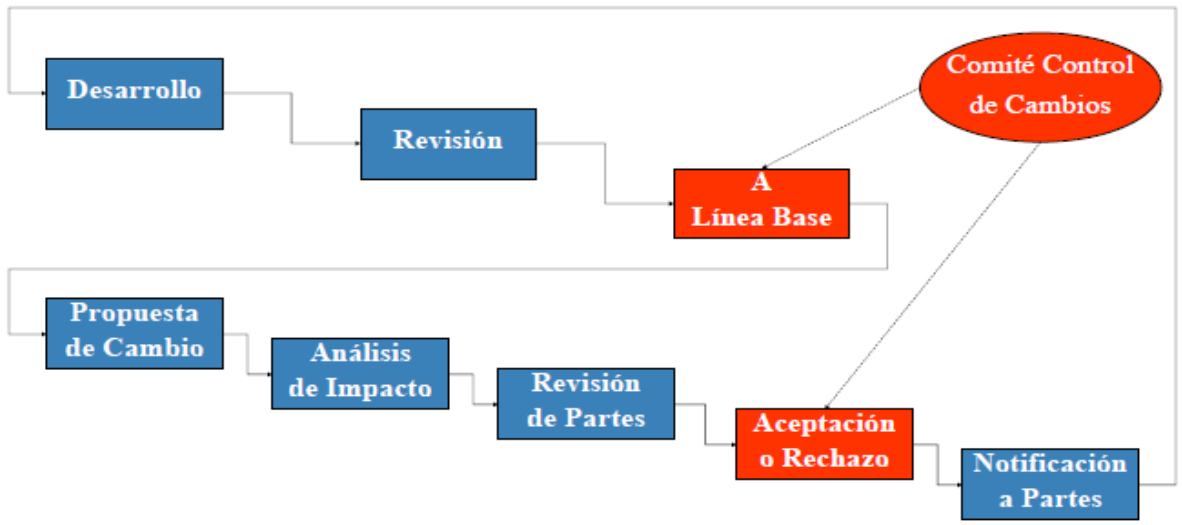
Implica una identificación unívoca para cada ítem, donde en el equipo se definirán políticas y reglas de nombrado y versionado para todos ellos. También, se debe definir la estructura del repositorio, y la ubicación de los ítems de configuración dentro de esa estructura.
Esta identificación y documentación proveen un camino que une todas las etapas del ciclo de vida del software, lo que permite a los desarrolladores controlar y velar por la integridad del producto, como así también a los clientes evaluar esa integridad.

- **Ítems de producto:** Tienen el ciclo de vida más largo, y se mantienen mientras el producto exista. Ejemplo: documento de arquitectura, casos de uso, código, manual de usuario y de parametrización, casos de prueba, una ERS, los casos de prueba, la base de datos.
- **Ìtems de proyecto:** El plan de proyecto, el listado de defectos encontrados, tienen un ciclo de vida de proyecto. Un plan de iteración, un Burn-Down Chart, se mantiene durante una iteración. La duración impacta también en el esquema de nombrado, plan de proyecto. Ejemplo: El plan de proyecto y el cronograma.
- **Ítems de iteración:** Conocer el ciclo de vida de un ítem permite establecer la nomenclatura de este. Ejemplo: planes de iteración, cronogramas de iteración, reporte de defectos.

Control de cambios

Busca mantener la integridad y trazabilidad de las líneas base (LB). Para cambiar la LB, esta debe pasar por un proceso formal de control de cambios. En base a esto decimos que los IC que NO son LB se pueden modificar sin tanto problema. Analizaremos sus etapas:

1. Se recibe una propuesta de cambio sobre una línea base determinada, no sobre un ítem.
2. El comité de control de cambios (formado por representantes de todas las áreas involucradas en el desarrollo) realiza un análisis de impacto del cambio para evaluar el esfuerzo técnico, el impacto en la gestión de los recursos, los efectos secundarios y el impacto global sobre la funcionalidad y la arquitectura del producto.
3. En caso de que se autorice la propuesta de cambio, se genera una orden de cambio que define lo que se va a realizar, las restricciones a tener en cuenta y los criterios para revisar y auditar.
4. Luego de realizado el cambio, el comité vuelve a intervenir para aprobar la modificación de la línea base y marcarla como línea base nuevamente, es decir que realiza una revisión de partes.
5. Finalmente se notifica a los interesados los cambios realizados sobre la línea base.



Auditorías de configuración

Son controles autorizados a realizar por el equipo de desarrollo en un momento determinado, donde un auditor externo al equipo (independiente y objetivo) analiza las líneas base, las cuales permiten "congelar" y analizar en un momento determinado cuál es el estado del software, y si se están cumpliendo todas las pautas que plantea esta disciplina de SCM.
En metodologías ágiles, esta es la única actividad de la disciplina SCM que no se soporta por la filosofía. Es objetiva si hay independencia de criterio y el auditor no tiene ningún condicionamiento, y es independiente cuando no depende jerárquica, funcional ni salarialmente del equipo o elemento a auditar.
La auditoría de configuración se hace mientras el producto se está construyendo y su objetivo es que el producto tenga calidad e integridad. Se entiende que lo más barato es prevenir. Complementa a la revisión técnica y consiste en revisar si se están realizando las tareas tales como se planificaron y especificaron en el plan de gestión de configuración.

Genera trazabilidad que permite realizar un análisis del impacto sobre el sistema a partir de una solicitud de cambio en el requerimiento. Los modelos de calidad exigen la trazabilidad bidireccional. Desde el requerimiento hacia el componente de código y viceversa.

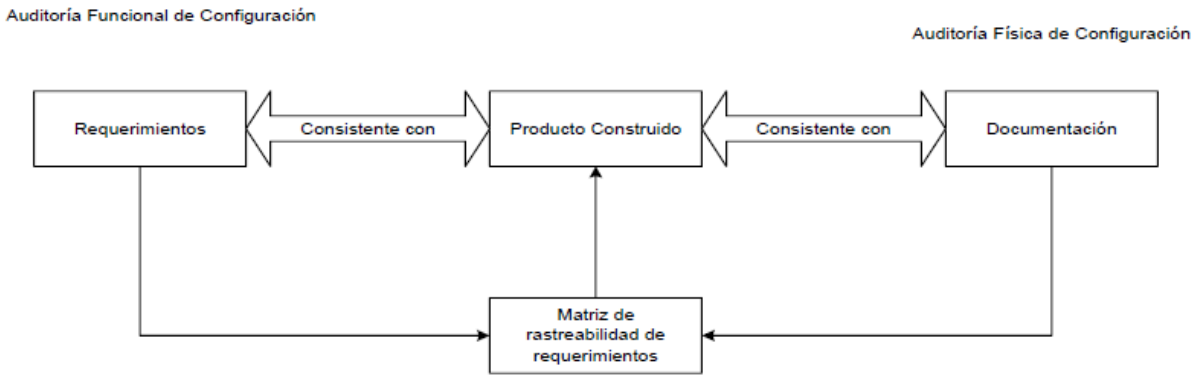
Procesos a los que sirve:

- **Validación:** Se encarga de asegurar que cada ítem de configuración resuelva el problema apropiado, es decir, lo que el cliente necesita.
- **Verificación:** Implica asegurar que un producto cumple con los objetivos definidos en la documentación de líneas base. Todas las funciones son llevadas a cabo con éxito y los test cases tengan status "ok" o bien consten como "problemas reportados" en la nota de reléase. Es decir, que se cumpla lo definido para cada ítem de configuración en la documentación de una línea base.

Tipos de auditorías:

- **Auditoría física o de contingencia:** Se verifica primero. Se validan los ítems de configuración de la línea base contra el plan de gestión de configuración, asegurando que la documentación que se entregará es consistente con el código desarrollado. Si esto no pasa, la auditoría funcional no se realiza. Controla que el repositorio esté alojado en el lugar donde se dijo que iba a estar, que los IC respeten el esquema de nombrado, que estén guardados donde se dijo que iban a estar guardados. Lo que se audita es una línea base. Hace verificación.
- **Auditoría funcional de configuración:** Audita la trazabilidad de requerimientos, controlando que la funcionalidad y performance real de cada ítem de configuración sean consistentes con la especificación de requerimientos. El propósito final es validar que el código implementa únicamente y todos los requerimientos de la ERS. Evaluación independiente de los productos de software, controlando que la funcionalidad y performance reales de cada ítem de configuración sean consistentes con la especificación de requerimientos. Antes de comenzar, pregunta si está disponible el reporte de la auditoría de configuración física. Hace validación.

La gestión en ambientes ágiles se trabaja con integración continua (CI - Continuous Integration), un servidor donde se testea automáticamente el código y luego se integra en producción continuamente. Un repositorio, un esquema de nombrado son cosas que si se utiliza en ágiles, mientras que la auditoría y el comité de control de cambios durante el sprint, pueden obviarse



Informe de estado

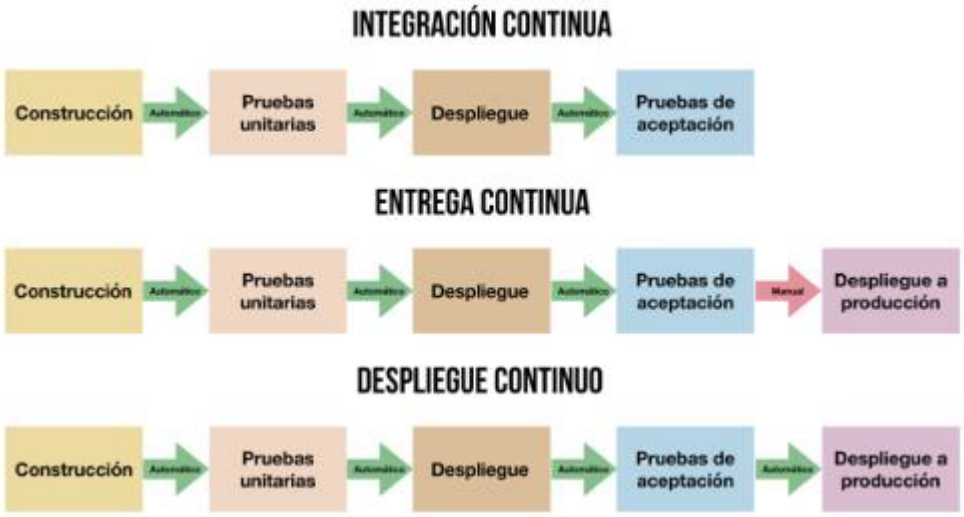
Su propósito es generar reportes para mantener un registro de la evolución, para dar visibilidad para la toma de decisiones. Sirve para que los involucrados se enteren de un estado de situaciones de la gestión de configuración.
El reporte básico es el inventario, que lista todos los IC que tiene el repositorio con su estado en un momento de tiempo o listar cuántas solicitudes de cambios se atendieron en un determinado periodo de tiempo. La mayoría de los reportes se obtienen de manera automática.
Registro e informe de estado:

- Se ocupa de mantener los registros de la evolución del sistema.
- Maneja mucha información y salidas por lo que se suele implementar dentro de procesos automáticos.
- Incluye reportes de rastreabilidad de todos los cambios realizados a las LB durante el ciclo de vida.

Plan de gestión de configuración: El plan debe contener respuestas a las preguntas de cómo voy a hacer las actividades básicas de la gestión de configuración, cuanto a auditorías y de qué tipo y en qué momento voy a hacer, que informes de estado voy a sacar, la estructura de repositorio, cómo se conforma el comité de control de cambios.

Evolución de la gestión de configuración de software

El SCM permitió que los equipos evolución en su forma de construir sw y que traten de hacer un proceso que sea cada vez más productivo y para logarlo se tratará de automatizar la mayor parte que se puedan de los procesos.



Estas disciplinas son la evolución de la gestión de configuración de software, si no esta la base de gestión de configuración como primera capa ninguna de estas cosas es posible.
Las metodologías ágiles, a diferencia de lo que todo el mundo cree, formalizan mucho y son muy rigurosas respecto a la calidad y a la forma en la que se va a construir el código. Se brinda mucha importancia al testing y que esté lo más automatizado posible.
La integración continua, que es lo primero que se hacer, es que cada desarrollados en su entorno trabaja, realiza pruebas unitarias, desarrollando con algo que se llama TDD (Desarrollo conducido por testing) al terminar ese componente de código y lo probó y sabe que funciona, se sube a un repositorio de integración.
La entrega continua suma la automatización de las pruebas de aceptación y entonces el producto ya está listo para desplegarlo a producción.
Cuando el último paso también es automatizado, tenemos finalmente el despliegue continuo, que consiste en poner en el ambiente de producción del usuario final el producto.
“La diferencia entre estas tres prácticas es el nivel de automatización de las pruebas.”

Gestión de configuración en ambientes ágiles

En las metodologías ágiles la gestión de configuración cambia el enfoque, ya que la misma es de utilidad para los miembros del equipo de desarrollo y no viceversa.
En orden con la caracterización de los equipos ágiles, decimos que la gestión de configuración posibilita el seguimiento y la coordinación del desarrollo en lugar de controlar a los desarrolladores.

Los equipos ágiles son autoorganizados, por lo que las Auditorías de configuración no son una actividad propia de la gestión de configuración en los ambientes ágiles. Todos los procesos definidos establecidos en la gestión de configuración del enfoque tradicional son relativizados en agile. Por ejemplo, no existe un comité para el proceso forma de control de cambios.

Podemos decir que va de la mano con el manifiesto ágil porque recordemos que se buscaba estar siempre preparados para el cambio y justamente, la gestión de configuración busca responder a los cambios y mantener la integridad del producto.
La diferencia es que el manifiesto ágil se enfoca en los proyectos, mientras que la gestión de configuración de software se enfoca en el producto. Es decir, trasciende el proyecto. Mientras el manifiesto ágil apunta a cómo trabajar en el contexto del proyecto de desarrollo y la gestión de configuración es una práctica específica del producto que garantiza la calidad del producto. **Características:**

- Dada la naturaleza de los requerimientos ágiles, la SCM responde a los cambios en lugar de evitarlos.
- La automatización debe ser aplicada donde sea posible. Esto colabora con la entrega frecuente y rápida de software (integración continua).
- Provee retroalimentación continua sobre calidad, estabilidad e integridad.
- Las actividades de SCM se encuentran embebidas en las demás tareas para alcanzar el objetivo del sprint.
- Es responsabilidad de todo el equipo, por lo tanto, requiere capacitación.
- Adopción de las prácticas continuas.